



Cairo University
Faculty of Computers and Artificial Intelligence
Operations Research and Decision Support Department

Egyptian Premier League Schedule Optimizer

Graduation Project — Final Documentation

Submitted to the Faculty of Computers and Artificial Intelligence, Cairo University,
In Partial Fulfillment of the Requirements for the bachelor's degree in
Operations Research and Decision Support

Academic Year 2025–2026

Under Supervision of:

Dr. Sally Kassem

Eng. Rawaa Hesham

Rawan Ehab	20220133
Mohamed Osama	20220477
Abd El-Rahman Ashraf	20220189
Ghassan Tarek	20220239
Ibrahim Medhat	20221003

ABSTRACT

The Egyptian Premier League scheduling process has evolved over time and is no longer entirely manual, and already considers multiple operational factors such as competition calendars, stadium availability, and security requirements. However, despite these improvements, the scheduling process still faces recurring challenges, including frequent postponements, uneven rest periods between teams, congestion caused by overlapping competitions, and difficulties in maintaining a standard league format within a fixed seasonal timeframe. These issues indicate that the existing scheduling approach lacks a unified optimization framework capable of evaluating all constraints and objectives simultaneously.

This project presents an optimization-based scheduling system using Constraint Programming with SAT solving (CP-SAT), implemented via Google OR-Tools to enhance the current scheduling process rather than replace it. The league season is modeled as a constrained optimization problem where matches are assigned to predefined calendar slots and venues while respecting competition rules, continental commitments, security constraints, and fairness considerations. The system was implemented using Python, where structured data inputs are combined with mathematical constraints and objective functions to systematically evaluate scheduling alternatives.

The optimizer minimizes a weighted penalty covering round-order deviation, weekly load balance, away-team travel distance, and match-tier vs. slot-tier alignment — ensuring high-profile fixtures are placed in prime-time broadcast slots. A dedicated post-baseline CAF audit and greedy repair phase detects and reschedules any matches that violate continental competition buffer rules.

TABLE OF CONTENTS

Contents

ABSTRACT.....	2
TABLE OF CONTENTS	3
LIST OF TABLES	7
CHAPTER 1	1
INTRODUCTION TO THE PROJECT REPORT AND PROJECT OVERVIEW	1
1 INTRODUCTION	2
1.1 PROBLEM DOMAIN.....	2
1.2 PROBLEM STATEMENT	3
1.3 PROPOSED SYSTEM.....	4
1.3.1 AIMS AND OBJECTIVES	4
1.3.2 PROPOSED SYSTEM FEATURES.....	5
1.4 DEVELOPMENT METHODOLOGY	6
1.5 RESOURCE REQUIREMENT	6
1.6 REPORT LAYOUT	7
CHAPTER 2	8
BACKGROUND/EXISTING WORK.....	8
2 BACKGROUND	9
2.1 INTRODUCTION TO SPORTS SCHEDULING	9
2.2 THEORETICAL CONCEPTS	9
2.2.1 PATTERNS AND BREAKS.....	9
2.2.2 THE TRAVELING TOURNAMENT PROBLEM (TTP).....	10
2.3 MATHEMATICAL APPROACHES.....	10
2.3.1 INTEGER LINEAR PROGRAMMING (ILP).....	10
2.4 REVIEW OF EXISTING REAL-WORLD IMPLEMENTATIONS.....	10
2.4.1 THE CHILEAN FIRST DIVISION (THE "GOLD STANDARD").....	10
2.5 THE EGYPTIAN CONTEXT: GAPS IN CURRENT PRACTICE	11
CHAPTER 3A.....	12
PROJECT DATA AND DATA PREPROCESSING PHASES	12
3 PROJECT DATA AND DATA PREPROCESSING	13
3.1 DATA COLLECTION.....	13
3.2 DATA STRUCTURE	14
3.2.1 TEAMS DATA	14
3.2.2 STADIUM DATA.....	15
3.2.3 CALENDAR DATA	15
3.2.4 SECURITY MATRIX.....	16
3.2.5 CONTINENTAL (CAF) SCHEDULE.....	17

3.2.6	DISTANCE MATRIX SCHEMA	17
3.2.7	HISTORICAL BENCHMARK DATASET	18
3.3	DATA PREPROCESSING	19
CHAPTER 3B		21
MATHEMATICAL FORMULATION		21
3.4	SETS	22
3.5	PARAMETERS	22
3.5.1	MATCH PARAMETERS	22
3.5.2	SLOT PARAMETERS	23
3.5.3	ROUND PARAMETERS	23
3.5.4	CAF PARAMETERS	23
3.5.5	CAPACITY CONSTANTS	23
3.5.6	VENUE PARAMETERS	24
3.6	FEASIBLE DOMAIN	24
3.7	DECISION VARIABLES	24
3.7.1	PRIMARY ASSIGNMENT	24
3.7.2	AUXILIARY TIME VARIABLES	25
3.7.3	WEEKLY LOAD VARIABLES	25
3.7.4	FINAL-ROUND SLOT VARIABLES	25
3.7.5	COLLISION INDICATOR VARIABLES	25
3.8	HARD CONSTRAINTS	26
3.9	OBJECTIVE FUNCTION	30
3.10	OBJECTIVE WEIGHTS	32
3.11	MODEL SUMMARY	32
3.11.1	HARD CONSTRAINTS	32
3.11.2	OBJECTIVE TERMS	33
SYSTEM DESIGN		34
4	SYSTEM ARCHITECTURE AND COMPONENT OVERVIEW	35
4.1	LAYERS	35
4.2	COMPONENT DIAGRAM	36
4.3	DEPLOYMENT ARCHITECTURE	36
4.4	CLASS / MODULE DIAGRAMS	37
4.4.1	KEY DATA STRUCTURES USED BY THE FINAL PIPELINE	38
	• MATCH (SRC/FIXTURE_GENERATOR.PY)	38
	• ROUNDWINDOW (SRC/SLOT_DOMAIN.PY)	38
	• SCHEDULEDMATCH (SRC/BASELINE_SOLVER.PY)	38
	• RUNMETRICS (SRC/MULTI_RUN.PY)	38
4.5	SEQUENCE DIAGRAMS (KEY USE CASES)	39
4.5.1	USE CASE: RUN THE FULL PIPELINE FROM THE STREAMLIT UI	39
4.5.2	USE CASE: EXPLORE A GENERATED SCHEDULE (TEAM CHOOSER)	40
4.5.3	USE CASE: INSPECT SCHEDULE QUALITY (INSIGHTS TAB)	40
5	I IMPLEMENTATION ENVIRONMENT AND SETUP	43
5.1	LANGUAGE, RUNTIME, AND CORE LIBRARIES	43
5.2	PROJECT LAYOUT	44
5.3	RUNNING THE PROJECT	44
5.4	KEY IMPLEMENTATION DETAILS	45

5.4.1	FIXTURE GENERATION: CIRCLE METHOD + CP-SAT ORIENTATION	45
5.4.2	SLOT DOMAINS AND THE CAF REST-DAY RULE	45
5.4.3	THE BASELINE CP-SAT MODEL	46
5.4.4	DOMAIN-FALLBACK RETRIES AND THE FINAL-ROUND RESCUE MODEL	47
5.4.5	POST-SOLVE VALIDATION	48
5.4.6	STREAMLIT APPLICATION: CACHING AND NON-BLOCKING RUNS	48
5.5	SAMPLE SCREENSHOTS / OUTPUT SAMPLES	49
		52
5.6	FINAL SCHEDULE OUTPUT (OPTIMIZED_SCHEDULE.CSV)	52
6	TESTING	54
6.1	TESTING STRATEGY	54
6.1.1	BUILD-TIME STRUCTURAL ASSERTIONS	54
6.1.2	AUTOMATED POST-SOLVE VALIDATION	54
6.1.3	MANUAL / EXPLORATORY TESTING VIA THE DASHBOARD	55
6.2	TEST CASES AND RESULTS	55
6.3	PERFORMANCE EVALUATION / BENCHMARKING	57
6.3.1	SOLVER PERFORMANCE	57
6.3.2	EVALUATING TRAVEL EFFICIENCY	58
6.3.3	STADIUM UTILIZATION AND MAINTENANCE OVERLAPS	58
6.3.4	BENCHMARKING AGAINST HISTORICAL SEASONS	60
6.3.5	BENCHMARKING ACROSS SEEDS (MONTE CARLO)	61
6.4	LIMITATIONS AND KNOWN ISSUES	61
6.4.1	TESTING-PROCESS LIMITATIONS	61
6.4.2	APPLICATION LIMITATIONS	61
7	CONCLUSION	63
7.1	SUMMARY OF ACHIEVEMENTS	63
7.2	LESSONS LEARNED	64
7.2.1	THE FINAL ROUND IS A SOURCE OF DIFFICULTY	64
7.2.2	REAL DATA IS MESSIER THAN THE MODEL ASSUMES	64
7.3	FUTURE ENHANCEMENTS	64
7.3.1	MULTI-OBJECTIVE PARETO OPTIMIZATION	64
7.3.2	METAHEURISTIC HYBRIDIZATION	65
7.3.3	WEATHER AND RAMADAN AWARENESS	65
7.3.4	SENSITIVITY ANALYSIS	65
7.3.5	API LAYER	65
7.3.6	LEAGUE-AGNOSTIC CONFIGURATION	65
7.3.7	EXPORT TO ICAL/GOOGLE CALENDAR	65
8	REFERENCES	66

LIST OF FIGURES

Figure 1: Team & Stadiums Relation Diagram	19
Figure 2: Dates Relationship Model	20
Figure 3 : System Component Diagram	36
Figure 4 : Class Diagram	37
Figure 5: Full Pipeline Use Case	39
Figure 6: Generated Schedule Use Case	40
Figure 7: Explore Insights Use Case.....	41
Figure 8: Sidebar	49
Figure 9: “Run & progress” tab mid-run	50
Figure 10: “Explore” tab — Team chooser	50
Figure 11: “Insights” tab — Overview sub-tab.....	51
Figure 12: “Explore” tab — Calendar sub-tab.....	51
Figure 13: Match Details Card.....	52
Figure 14: Output Format	52
Figure 15: Results Overview.....	57
Figure 16: Travel Fairness Insights	58
Figure 17: Stadium Service Gaps	59
Figure 18: Venue Utilization Timeline.....	59
Figure 19: Matches per Venue:.....	59
Figure 20: The 'Ghost Gap' Comparison	Figure 21: Venue Symmetry (Fairness)
Figure 22: full historical audit trail table	60
Figure 23: Monte-Carlo results	61

LIST OF TABLES

Table 1: Teams Dataset Schema	14
Table 2: Stadiums dataset schema	15
Table 3: Calendar data schema	16
Table 4: Security Constraints Schema.....	16
Table 5:Continental (CAF) Schedule Schema	17
Table 6: Distance Matrix Schema.....	17
Table 7: Past Data schema	18
Table 8: Sets.....	22
Table 9: Match Parameters	22
Table 10: Slot Parameters	23
Table 11: Round Parameters	23
Table 12: CAF Parameters.....	23
Table 13: Capacity Constants	23
Table 14: Objective Weights	32
,Table 15: Hard Constraints	33
Table 16: Objectives	33
Table 17: Used Libraries.....	43
Table 18: Test Cases	56

CHAPTER 1

INTRODUCTION TO THE PROJECT REPORT AND PROJECT OVERVIEW

1 Introduction

Football competitions are complex systems that require careful planning and coordination between many parties, including teams, stadiums, broadcasters, and governing bodies. The Egyptian Premier League is one of the most followed sports competitions in the country and involves many matches played over a long season. Creating a league schedule that satisfies all operational requirements while maintaining fairness between teams is a challenging task known in the field of Operations Research as the "Sports Scheduling Problem" [1].

In recent years, the league scheduling process has improved and now considers several important factors such as competition calendars, stadium availability, and security requirements, notably through partnerships with specialized scheduling firms [2]. However, despite these developments, league seasons continue to suffer from delays, congestion, and frequent rescheduling due to external factors like CAF interruptions. These issues suggest that while the scheduling process exists, it is not fully optimized and does not always provide the best possible outcomes under complex constraints.

This project focuses on enhancing league scheduling using mathematical optimization techniques. By modeling the Egyptian Premier League as an optimization problem, we aim to generate a complete and balanced schedule that respects all constraints while fitting within a fixed seasonal timeframe. It demonstrates how structured optimization models can support decision-making in sports scheduling and improve the overall organization of league competitions.

1.1 Problem Domain

The problem domain of this project falls under sports scheduling optimization, which is a subfield of operations research and applied mathematics. Sports scheduling problems involve assigning matches to time slots and venues while respecting a wide range of constraints related to logistics, fairness, and competition rules.

In the context of the Egyptian Premier League, scheduling must account for domestic league matches, cup competitions, continental tournaments organized by CAF, international breaks, stadium sharing, and security considerations. Each of these factors imposes limitations on when and where matches can be played.

Additionally, teams differ in geographical location, stadium ownership, competitive importance, and participation in continental competitions. This leads to unequal travel demands and varying levels of schedule pressure across teams. Addressing these differences in a fair and systematic way places the scheduling problem firmly within the domain of constrained optimization rather than simple timetable generation.

Because of the large number of variables and constraints involved, this problem is well-suited for mathematical modeling approaches, specifically Constraint Programming with SAT solving (CP-SAT) that can evaluate many possible schedules and identify solutions that best satisfy defined objectives [3].

1.2 Problem Statement

Although the Egyptian Premier League scheduling process considers multiple constraints, it continues to experience structural issues that affect competition quality and fairness. The roots of this chronic instability can be traced back to the 2018/2019 season. When Egypt was selected as the emergency host for the 2019 African Cup of Nations (AFCON) replacing Cameroon, the domestic league was forced into a severe hiatus to facilitate stadium preparations. This disruption was soon followed by the global COVID-19 pandemic, which forced a four-month suspension of all football activities in 2020. The cumulative impact of these two consecutive shocks created a "domino effect" of delayed start dates and congested fixtures that the league has yet to fully recover from [4].

The core problem is the absence of a unified optimization framework that simultaneously considers all scheduling constraints and objectives. Existing approaches tend to address constraints individually, which can result in local solutions that create larger conflicts later in the season. As a result, league organizers are frequently forced to introduce exceptional arrangements that reduce competitive balance.

The problem addressed in this project is how to construct a complete Egyptian Premier League schedule that satisfies operational constraints, improves fairness, and fits within a fixed August-to-May season without modifying the league structure. The project investigates whether an optimization-based approach can systematically improve scheduling outcomes compared to existing practices

1.3 Proposed System

The proposed system is an optimization-based league scheduling model developed using **Constraint Programming with SAT solving (CP-SAT) via the Google OR-Tools library**. Rather than replacing existing scheduling practices, the system aims to formalize and enhance them by introducing a structured mathematical framework that evaluates all constraints and objectives simultaneously.

The league season is represented as a set of decision variables that determine whether a specific match is assigned to a particular time slot and venue. Constraints are defined to enforce league rules, competition calendars, stadium availability, security restrictions, and rest requirements. An objective function is then used to guide the model toward schedules that maximize fairness and operational efficiency.

The system is implemented using Python, where the Google OR-Tools CP-SAT library are used to define the model structure. External solvers are employed to efficiently explore possible scheduling combinations and identify feasible solutions. The final output of the system is a complete season schedule accompanied by performance statistics that allow evaluation of schedule quality.

1.3.1 Aims and Objectives

The main aim of this project is to demonstrate the effectiveness of mathematical optimization techniques in improving professional football league scheduling. The project seeks to show that a standard league format can be maintained when scheduling decisions are supported by structured optimization models.

The specific objectives of the project are:

- To model the Egyptian Premier League scheduling problem using CP-SAT constraint programming.
- To generate a complete 18-team double round-robin league schedule.
- To ensure the season fits within the August to May timeframe.
- To respect continental competitions, domestic cups, and international breaks.
- To avoid stadium and security conflicts.
- To improve fairness by balancing home and away matches and rest periods.
- Reducing excessive travel demands where possible.
- To align high-profile matches with prime-time broadcast slots using the tier-slot matching objective.
- To evaluate schedule quality using measurable performance indicators.

1.3.2 Proposed System Features

The proposed system includes several features designed to improve scheduling outcomes. It automatically handles stadium conflicts by shifting match times or assigning alternative venues when required. It also blocks league matches during periods reserved for continental competitions or cup rounds.

A key feature is the three-tier slot classification system: Tier-1 slots are Friday/Saturday evenings ($\geq 19:00$); Tier-2 are weekend afternoons or weekday evenings; Tier-3 are weekday afternoons. A corresponding match-tier is computed from team quality. The solver hard-constrains Tier-1 matches to Tier-1 slots and softly penalizes any tier mismatch.

Fairness-related features include limiting consecutive away matches and ensuring that the final match day is consolidated to a single calendar date with kickoff times marked TBD, preserving the tradition of simultaneous final-round fixtures.

The system also includes a dedicated post-baseline CAF audit and greedy repair phase. After the main solver produces a baseline schedule, a separate audit scans every match

involving CAF-participating teams, identifies violations of the 4-day buffer rule, and invokes a greedy repair loop to reschedule those matches into valid slots without disturbing accepted fixtures.

Another key feature is adaptability. When external conditions change, such as updates to competition calendars or stadium availability, the system can be re-run using updated data to produce a revised schedule within a short time.

1.4 Development Methodology

The development of the project follows a structured methodology. The first stage involves analyzing the league structure and identifying all relevant constraints and data requirements. This is followed by the design of input data sheets representing teams, stadiums, calendar slots, security restrictions, and continental commitments.

Next, the mathematical model is to be formulated using Constraint Programming (CP-SAT). Decision variables are defined to represent match assignments, and constraints are added incrementally to ensure correctness. With an objective function constructed to reflect fairness and efficiency goals. The full model formulation is presented in Chapter 3b.

The model is implemented in Python using OR-Tools, and multiple test scenarios are to be executed to validate feasibility and performance. The resulting schedules are analyzed using statistical measures to assess fairness, travel, and constraint satisfaction.

1.5 Resource Requirement

The project requires a limited set of resources. Software requirements include Python, spreadsheet tools for data preparation, and the Google OR-Tools CP-SAT solver (or-tools Python package).

These tools are sufficient to model and solve the scheduling problem efficiently. A standard personal computer with sufficient RAM is adequate for running the optimization model within the configured time limits.

1.6 Report Layout

This report is organized into several chapters. Chapter **One** introduces the project, defines the problem domain, and presents the proposed system. Chapter **Two** reviews project background and existing work.

Chapters **3, 4, 5** describe the system design and mathematical model in detail, including decision variables, constraints, and objective functions. They also include validation and testing of created schedules compared to old schedules.

Chapters **6, 7** explains the output and discusses features and limitations that were beyond project scope that could be done in future work.

CHAPTER 2

BACKGROUND/EXISTING WORK

2 Background

2.1 Introduction to Sports Scheduling

Sports scheduling is a specialized domain within Operations Research (OR) and Combinatorial Optimization. While it may appear to be a simple administrative task, mathematically, it is classified as an **NP-hard** problem, meaning the computational complexity increases exponentially with the number of teams and constraints [1].

In a standard football league, the core structure is the **Double Round Robin (DRR)** tournament. In a DRR with n teams, every team must play every other team twice—once at home and once away. This results in a total number of matches calculated as $n(n-1)$. For an 18-team league like the Egyptian Premier League, this equates to 306 total matches played over 34 rounds (or "weeks").

The goal of scheduling is not merely to find *any* valid schedule, but to find an *optimal* one that satisfies a set of rigorous constraints (rules) while optimizing specific objectives (such as minimizing travel or maximizing fairness) [5].

2.2 Theoretical Concepts

To understand the complexity of the problem, several key theoretical concepts from existing literature must be defined.

2.2.1 Patterns and Breaks

A "pattern" represents the sequence of home (H) and away (A) games for a specific team. For example, a pattern H-A-H-A is considered perfect alternation. A "**Break**" occurs when a team plays two consecutive matches at the same venue (e.g., H-H or A-A).

- **Home Break:** Playing two home games in a row. This is generally desirable for revenue but can lead to "clumping" of difficult fixtures.
- **Away Break:** Playing two away games in a row. This is considered undesirable due to travel fatigue and lack of revenue continuity. Literature suggests that minimizing the total number of breaks across the league is a primary objective for fairness [8].

In their survey of 25 European leagues, Goossens and Spokesman (2012) found that minimizing breaks is a universal objective. However, they noted that perfect alternation (H-A-H-A) is mathematically impossible for all teams simultaneously in standard league formats, necessitating optimization models to distribute these breaks fairly [6].

2.2.2 The Traveling Tournament Problem (TTP)

A major focus of existing work is the Traveling Tournament Problem (TTP), first defined by Easton et al. (2001). The TTP aims to minimize the total distance traveled by all teams during the season. This is particularly critical in large countries (like the USA or Brazil) or leagues where teams are geographically dispersed. While Egypt's teams are concentrated around Cairo and Alexandria, travel to Upper Egypt (e.g., Aswan) or canal cities (e.g., Ismailia) still presents a logistical cost that optimization models seek to balance [7].

2.3 Mathematical Approaches

Over the past two decades, the transition from manual to automated scheduling has been driven by advances in Integer Linear Programming (ILP).

2.3.1 Integer Linear Programming (ILP)

ILP is the most widely adopted method for professional sports scheduling. It involves defining the schedule as a set of linear equations where variables must be integers (0 or 1).

Decision Variable: $X_{i,j,k} = 1$ if team i plays team j in round k , and 0 otherwise.

Constraints: Mathematical inequalities that enforce rules.

Objective Function: A mathematical formula to minimize a penalty score (e.g., "Minimize total travel distance" or "Minimize security conflicts").

2.4 Review of Existing Real-World Implementations

2.4.1 The Chilean First Division (The "Gold Standard")

One of the most successful applications of sports scheduling optimization occurred in Chile. In 2005, the National Association of Professional Football (ANFP) replaced their manual system (which used a random draw) with an ILP model developed by the University of Chile.

- **Problem:** The manual schedule resulted in high travel costs and unfair sequences where small teams faced "giants" (top teams) consecutively.
- **Solution:** The new system considered geography, stadium capacities, and TV requirements.
- **Outcome:** The implementation of an optimization-based scheduling system in the Chilean League directly resulted in a **32–39%** increase in average attendance and a near-doubling of ticket revenue by strategically placing high-profile matches later in the season and optimizing fixtures for tourist regions [8].

2.5 The Egyptian Context: Gaps in Current Practice

In the context of the Egyptian Premier League, the scheduling approach has historically been manual or semi-automated, managed by the Competitions Committee. Unlike the European examples above, the Egyptian league faces unique "hard constraints" that standard commercial software often fails to address:

- **Security Approvals:** Match venues and dates are subject to approval by security authorities, often requiring last-minute changes.
- **Stadium Sharing:** A significant number of teams select the **Cairo International Stadium** or **Borg Al-Arab** as their home venue, creating a bottleneck that does not exist in leagues where every team owns its stadium.
- **CAF Competition Uncertainty:** The scheduling of Al Ahly and Zamalek in the CAF Champions League often forces indefinite postponements, breaking the rhythm of the standard mathematical "patterns" discussed in Section 2.2.1.

CHAPTER 3A

PROJECT DATA AND DATA PREPROCESSING PHASES

3 Project data and data preprocessing

For the Egyptian Premier League, the system requires detailed information about teams, stadiums, and the season calendar. This chapter describes how the data was obtained, how it is organized, and the steps taken to prepare it for the optimization system.

3.1 Data Collection

The project data was collected using Python-based web scraping scripts. These scripts were designed to automatically extract structured information from websites such as “Yallakora” and “Transfermarkt”.

The final dataset is stored in two Excel workbooks: Data_Model.xlsx and expanded_calendar.xlsx. These are the authoritative inputs to the optimization pipeline. The data is divided into six primary tables:

1. **Teams:** Administrative details of the clubs.
2. **Stadiums:** Venue locations and capacities.
3. **Calendar:** The list of available dates and time slots.
4. **Security Matrix:** A list of prohibited matchups and venue restrictions.
5. **Continental Blockers:** Schedules for African (CAF) competitions.
6. **International Breaks:** A list of official FIFA breaks.

The extracted data was stored in tabular format and exported to spreadsheet files for further organization and integration. The use of automated scraping reduced manual data entry and ensured consistency with published football schedules.

Finally, to validate the performance of our optimization model, we collected **Historical Schedule Data** from previous seasons. This dataset acts as a 'Benchmark,' allowing us to compare our optimized schedule against the actual manual schedules used in the past to measure improvements in fairness and efficiency.

3.2 Data structure

To ensure the system works correctly, the data was organized into linked tables. Each table uses unique identifiers (IDs) to connect the information. The following sections describe the content of each input sheet.

3.2.1 Teams data

Attribute Name	Data Type	Description / Role
Team_ID	String	A unique identifier for the club (e.g., "AHL"). Used as the primary key.
Team_Name	String	The official display name of the club.
Home_Stadium	String	ID of the team's primary venue. Links to the Stadium dataset.
Alt_Stadium	String	ID of the backup venue used when the primary stadium is unavailable.
Tier	Integer	Classification rank (1–3). Used to balance fixture difficulty.
Gov_ID	String	The Governorate. Used to calculate travel distances in the objective function.
Cont_Flag	String	Continental status (e.g., "CL"). Flags teams that require international buffer days.

Table 1: Teams Dataset Schema

3.2.2 Stadium Data

Attribute Name	Data Type	Description / Role
Stadium_ID	String	Unique identifier for the venue (e.g., "CAIRO_INTL").
Name	String	The full display name of the stadium.
City	String	Geographic location.
Is_Floodlit	Boolean	Indicates if the stadium can host evening matches

Table 2: Stadiums dataset schema

3.2.3 Calendar Data

This sheet represents the time horizon for the season. It lists every possible "slot" where a match can be played.

Attribute Name	Data Type	Description / Role
Day_ID	String	Unique identifier for the date (e.g., D_20260801).
Date	Date	The calendar date (MM/DD/YYYY).
Date time	Time	Specific Kick-off Time. Defines the exact slot (e.g., 5:00:00 PM). Allows the model to schedule multiple non-overlapping matches on the same day.
Week_Num	Integer	The Game Week number (e.g., 1). Groups these slots into a specific match round.
Day_name	String	Day of the week (e.g., "SAT"). Used to prioritize Friday/Saturday slots.
Is_FIFA	Boolean	Blockage Flag: If 1, the slot is reserved for International Breaks.

Attribute Name	Data Type	Description / Role
Is_CAF	Boolean	Blockage Flag: If 1, the slot is reserved for Continental fixtures (Blocked only for CAF participants).
Is_Ramadan	Boolean	Constraint Flag: Indicates if the slot falls during Ramadan (later kick-off times).
Is_SuperCup	Boolean	Blockage Flag: If 1, the slot is reserved for the Egyptian Super Cup, pausing regular league play.

Table 3: Calendar data schema

3.2.4 Security Matrix

This table addresses the specific security constraints of the Egyptian League

Attribute Name	Data Type	Description / Role
Home_Team	String	ID of the hosting team involved in the restriction.
Away_Team	String	ID of the visiting team involved in the restriction.
Banned_Venue	String	ID of the specific stadium where match-up is forbidden.
Forced_Venue	String	Specifies a mandatory neutral venue for this match.

Table 4: Security Constraints Schema

3.2.5 Continental (CAF) Schedule

Attribute Name	Data Type	Description / Role
Date_Id	String	Unique ID linking this event to the main Calendar.
Competition_Name	String	CAF CL for Champions League, CAF CC for Confederation Cup).
Caf_Date	Date	The specific date of the international match.
Round	String	The stage of the competition
Team_Id	String	The specific club playing (e.g., AHL, ZAM). This allows the model to block <i>only</i> this team.
Location	String	"Home" or "Away". Used to calculate travel fatigue and buffer

Table 5: Continental (CAF) Schedule Schema

3.2.6 Distance Matrix Schema

Attribute Name	Data Type	Description / Role
Origin_Stadium	String	ID of the starting stadium (e.g., CAIRO_INTL).
Dest_Stadium	String	ID of the destination stadium (e.g., BORG_EL_ARAB).
Distance_KM	Float	The driving distance in Kilometers. Used to calculate the travel penalty in the Objective Function.

Table 6: Distance Matrix Schema

3.2.7 Historical Benchmark Dataset

Attribute Name	Data Type	Description / Role
Matchday	String	The round identifier (e.g., "1. Matchday"). Used to group historical matches.
Date	Date	The actual date the match was played in the past.
Home_Team	String	The name of the hosting club (Historical).
Away_Team	String	The name of the visiting club (Historical).
Time	Time	The historical kick-off time (e.g., "3:00 PM").

Table 7: Past Data schema

3.3 Data Preprocessing

Since the data was obtained from various external sources, a preprocessing phase was necessary to standardize the inputs and establish relationships between the different datasets.

- **Date Standardization:** The dates collected from different websites had inconsistent formats.
 - Dates were converted into a standard MM/DD/YYYY format to ensure compatibility with the system.
 - To ensure the tables were linked correctly, the Team IDs were checked across all sheets. For instance, inconsistencies in team names (e.g., "Al-Ahly" vs "Al Ahly SC") were corrected to a single code (AHL).
- **Relations and linking:** To ensure referential integrity between the disparate datasets, we constructed relational data. This transformed the flat Excel sheets into a connected schema.
 - **Infrastructure Model (Figure 1):** We established a Many-to-One relationship between the *Teams* table (Season_Team_Details) and the *Stadiums* table. Additionally, the Distance Matrix was linked to the Stadiums table, ensuring that every venue has a calculated travel cost to every other venue.



Figure 1: Team & Stadiums Relation Diagram

- **Temporal Model (Figure 2):** We created a "Star Schema" around the *Expanded_Calendar* table. By linking the *FIFA_DAYS* and *CAF_Dates* tables to the master *Day_ID*, the system can automatically flag specific time slots as "Blocked" without manual entry, ensuring that no domestic match is ever scheduled during a confirmed international window.

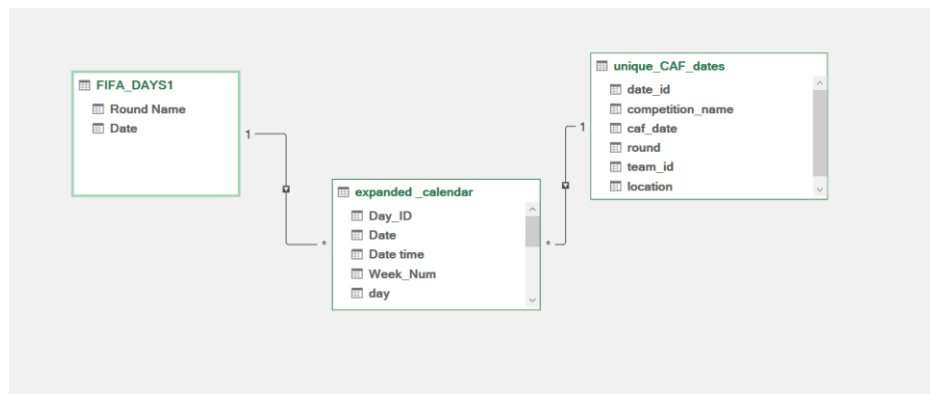


Figure 2: Dates Relationship Model

- **Data Integrity:** The data was largely used in its original form. Complex cleaning algorithms were not required; instead, the focus was on ensuring that the constraints (such as stadium capacities and security rules) were accurately represented in the Excel sheets.

CHAPTER 3B

MATHEMATICAL FORMULATION

3.4 Sets

Symbol	Definition
M	Set of all matches, $ M = 306$
S	Set of all usable (non-FIFA) calendar slots, $ S $ approx. 824
T	Set of all teams, $ T = 18$
V	Set of all stadiums, $ V = 22$
R	Set of round numbers $\{1, 2, \dots, 34\}$
$C \subseteq T$	Set of CAF-participating teams ($\text{Cont_Flag} \in \{\text{CL}, \text{CC}\}$)
$T1 \subseteq T$	Set of Tier-1 teams (league tier = 1)
W	Set of all calendar week numbers in the season
D	Set of all calendar dates in the season

Table 8: Sets

3.5 Parameters

3.5.1 Match parameters

Symbol	Definition
team(m)	Set contains the home and away team for match $m \in M$
home(m)	Home team of match m
away(m)	Away team of match m
tier_match(m)	Quality tier in $\{1,2,3\}$ of match m
round(m)	Abstract round number of match m in $\{1,\dots,34\}$
is_derby(m)	1 if home(m) in T1 and away(m) in T1, else 0 (needed by H2)

Table 9: Match Parameters

3.5.2 Slot parameters

Symbol	Definition
date(s)	Calendar date of slot s in S
week(s)	Calendar week number of slot s in S
tier_slot(s)	Quality tier in {1,2,3} of slot s (function of day-of-week and kickoff hour)
time_index(s)	Non-negative integer encoding the chronological position of slot s
kickoff_hour(s)	integer hour (0–23) of kickoff time of slot $s \in S$

Table 10: Slot Parameters

3.5.3 Round parameters

Symbol	Definition
nominal_week(r)	Ideal calendar week for round r, computed by linear interpolation across the season

Table 11: Round Parameters

3.5.4 CAF parameters

Symbol	Definition
caf_dates(t)	Sorted list of CAF match dates for team t in C
CAF_BUFFER	Minimum calendar days apart from any CAF date = $\text{MIN_REST_DAYS_CAF} + 1 = 4$

Table 12: CAF Parameters

3.5.5 Capacity constants

Constant	Value and meaning
MAX_DAY	3 max league matches on one calendar date
MAX_SLOT	2 max simultaneous matches in the same kickoff slot
HARD_MAX_WEEK	18 hard upper bound on matches per calendar week
SOFT_MIN_WEEK	6 soft lower targets for matches per calendar week
SOFT_MAX_WEEK	12 soft upper targets for matches per calendar week
REST_LOCAL	4 min calendar days between consecutive league matches
MAX_STREAK	2 max consecutive home (or away) matches

Table 13: Capacity Constants

3.5.6 Venue Parameters

Symbol	Definition
is_forced(m,v)	1 if venue v is security-mandated for match m, else 0
is_primary(m,v)	1 if v is the home team's primary stadium, else 0
is_alt(m,v)	1 if v is the home team's alternate stadium, else 0
is_other(m,v)	1 if v is neither primary nor alternate (and not banned), else 0
disp_km(m,v)	distance in km from the home team's primary stadium to venue v
travel_km(m,v)	distance from the away team's home stadium to venue v

Table 14: Venue Parameters

3.6 Feasible Domain

Let $\text{feasible_domain}(m)$ be the subset of S that match m is permitted to use before the solver runs. A slot s belongs to $\text{feasible_domain}(m)$ if and only if all the following hold:

(FD-1) $\text{date}(s)$ is not in FIFA_DATES

(FD-2) $\text{date}(s)$ has valid date, week, and kickoff metadata

(FD-3) For each t in $\text{team}(m) \cap C$, for each d in $\text{caf_dates}(t)$:

$$|\text{date}(s) - d| \geq \text{CAF_BUFFER} \quad (= 4 \text{ days})$$

Constraints H3-H11 are enforced inside the CP-SAT model and are not applied at domain-construction time.

3.7 Decision Variables

3.7.1 Primary assignment

$x[m,s,v] \in \{0,1\}$, for all $m \in M, s \in \text{feasible_domain}(m), v \in \text{VenueCandidates}$

$$x_{m,s,v} \in \{0,1\}, \quad \forall m \in M, s \in \text{feasible_domain}(m), v \in \text{VenueCandidates}(m)$$

- $x[m,s,v] = 1$ if and only if match m is assigned to calendar slot s at venue v .

3.7.2 Auxiliary time variables

$\text{match_time}[m]$ in $\mathbb{Z}^+$, for all m in M $\text{match_time}_m \in \mathbb{Z}^+, \forall m \in M$

$\text{round_min}[r]$ in $\mathbb{Z}^+$, for all r in R $\text{round_min}_r \in \mathbb{Z}^+, \forall r \in R$

$\text{round_max}[r]$ in $\mathbb{Z}^+$, for all r in R $\text{round_max}_r \in \mathbb{Z}^+, \forall r \in R$

3.7.3 Weekly load variables

load_w in $\mathbb{Z}^+$, for all w in W (total matches assigned to week w)

$\text{load_w} \in \mathbb{Z}^+, \forall w \in W$

under_load_w in $\mathbb{Z}^+$, for all w in W (matches below soft minimum)

$\text{under_load_w} \in \mathbb{Z}^+, \forall w \in W$

over_load_w in $\mathbb{Z}^+$, for all w in W (matches above soft maximum)

$\text{over_load_w} \in \mathbb{Z}^+, \forall w \in W$

3.7.4 Final-Round Slot variables

fr_slots in $\{0,1\}$, for all s in feasible_domain restricted to Round 34

$\text{fr_slots}_s \in \{0,1\}, \forall s \in \text{feasible_domain restricted to Round 34 matches}$

3.7.5 Collision Indicator Variables

slot_collisions in $\{0,1\}$ (1 if more than one match occupies slot s)

$\text{overlap}[v,d] \in \{0,1\}$ (1 if venue v has two non-forced matches within SERVICE_GAP days of date d)

$\text{slot_collision}_s \in \{0,1\}$

$\text{overlap}_{v,d} \in \{0,1\}$

3.8 Hard Constraints

H1 - Assignment uniqueness

$\text{Sum}_{s \text{ in feasible_domain}(m)} x[m,s] = 1, \text{ or all } m \text{ in } M$

$$\sum_s x_{ms} = 1 \quad \forall m \in M$$

- Every match is assigned to exactly one slot.

H2 - Tier-1 derby slot requirement

$\text{Sum}_{\{s \text{ in feasible_domain}(m), \text{ tier_slot}(s)=1\}} x[m,s] = 1,$

for all $m \text{ in } M$ such that $\text{is_derby}(m) = 1$

$$\sum_{s: \text{tier_slot}(s)=1} x_{ms} = 1 \quad \forall m : \text{is_derby}(m)=1$$

- A match between two Tier-1 teams must be scheduled in a Tier-1 (prime-time) slot.

H3 - Team date uniqueness

$\text{Sum}_{m: t \text{ in team}(m)} \text{Sum}_{s: \text{date}(s)=d} x[m,s] \leq 1,$

for all $t \text{ in } T, d \text{ in } D$

$$\sum_{m: t \in \text{team}(m)} \sum_{s: \text{date}(s)=d} x_{ms} \leq 1 \quad \forall t \in T, d \in D$$

- Each team plays at most one league match per calendar date.

H4 - Venue slot uniqueness

$\text{Sum}_{m \text{ in } M} x[m,s,v] \leq 1, \text{ for all } v \text{ in } V, s \text{ in } S$

$$\sum_{m \in M} x_{m,s,v} \leq 1, \quad \forall v \in V, \forall s \in S$$

- Each venue hosts at most one match per kickoff slot.

H5 - Daily match cap

$$\sum_m \sum_{s: \text{date}(s)=d} x[m, s] \leq \text{MAX_DAY}, \text{ for all } d \text{ in } D$$

$$\sum_m \sum_{s: \text{date}(s)=d} x_{ms} \leq \text{MAX_DAY} \quad \forall d \in D$$

- At most MAX_DAY = 3 league matches may be scheduled on any one calendar date.

H6 - Slot concurrency cap

$$\sum_m x[m, s] \leq \text{MAX_SLOT}, \quad \text{for all } s \text{ in } S$$

$$\sum_m x_{ms} \leq \text{MAX_SLOT} \quad \forall s \in S$$

- At most MAX_SLOT = 2 matches may share the same exact kickoff slot.

H7 - Team rest (sliding 4-day window)

$$\sum_{m: t \text{ in team}(m)} \sum_{s: \text{date}(s) \text{ in } [d, d+3]} x[m, s] \leq 1, \text{ for all } t \text{ in } T, \quad d \text{ in } D$$

$$\sum_{m: t \in \text{team}(m)} \sum_{s: \text{date}(s) \in [d, d+\text{REST_LOCAL}-1]} x_{ms} \leq 1 \quad \forall t \in T, d \in D$$

- Each team may play at most one league match in any 4-day window [d, d+3]. This guarantees a minimum of 3 full rest days between consecutive matches.

H8 - match_time definition

$$\text{match_time}[m] = \sum_{s \text{ in feasible_domain}(m)} \text{time_index}(s) * x[m, s], \text{ for all } m \text{ in } M$$

$$\text{match_time}_m = \sum_{s \in \text{feasible_domain}(m)} \text{time_index}(s) \cdot x_{ms} \quad \forall m \in M$$

$$\text{match_date}[m] = \sum_{s \text{ in feasible_domain}(m)} \text{date_index}(s) * x[m, s], \text{ for all } m \text{ in } M$$

$$\text{match_date}_m = \sum_{s \in \text{feasible_domain}(m)} \text{date_index}(s) \cdot x_{ms} \quad \forall m \in M$$

- Because exactly one x[m,s] equals 1, match_time[m] equals the chronological index of the chosen slot.

H9 - Round min/max definition

$\text{round_min}[r] = \min_{\{m: \text{round}(m)=r\}} \text{match_time}[m], \text{ for all } r$
 $\text{in } R$

$\text{round_max}[r] = \max_{\{m: \text{round}(m)=r\}} \text{match_time}[m], \text{ for all } r$
 $\text{in } R$

$\text{round_min}_r = \min_{m: \text{round}(m)=r} \text{match_time}_m \quad \forall r \in R$

$\text{round_max}_r = \max_{m: \text{round}(m)=r} \text{match_time}_m$

- Earliest and latest time-index for each round.

H10 - Strict round ordering

$\text{round_max}[r] + \text{MIN_ROUND_GAP} \leq \text{round_min}[r+1], \text{ for all } r$
 $\text{in } \{1, 2, \dots, 33\}$

$\text{round_max}_r + \text{MIN_ROUND_GAP} \leq \text{round_min}_{r+1}, \quad \forall r \in \{1, 2, \dots, 33\}$

- All matches of round r must finish strictly before the first match of round $r+1$. The constraint is enforced on date indices (one calendar-day gap), not raw time indices.

H11 - Weekly hard cap

$\sum_m \sum_{\{s: \text{week}(s)=w\}} x[m, s] \leq \text{HARD_MAX_WEEK}, \text{ for all } w$
 $\text{in } W$

$\sum_m \sum_{s: \text{week}(s)=w} x_{ms} \leq \text{HARD_MAX_WEEK} \quad \forall w \in W$

- At most $\text{HARD_MAX_WEEK} = 18$ league matches may be scheduled in any calendar week.

H12 - CAF buffer

$\text{Sum}_{\{s \text{ in feasible_domain}(m) : |\text{date}(s) - d| < \text{CAF_BUFFER}\}}$
 $x[m, s] = 0$, for all $m \text{ in } M$, $t \text{ in team}(m) \text{ intersect } C$, $d \text{ in caf_dates}(t)$

$$\sum_s : |\text{date}(s) - d| < \text{CAF_BUFFER} \quad x_{ms} = 0 \quad \forall m \in M, t \in \text{team}(m) \cap C, d \in \text{caf_dates}(t)$$

- A match involving a CAF-participating team may not be scheduled within $\text{CAF_BUFFER} = 4$ calendar days of that team's CAF match date. The buffer is bidirectional - it applies both before and after the CAF date.

H13 - Home/away streak limit

For all $t \text{ in } T$, for all $r \text{ in } \{1, \dots, 32\}$:

Number of consecutive home (or away) matches starting
 at round $r \leq \text{MAX_STREAK}$

$$\text{Streak}^h(t, r) \leq \text{MAX_STREAK} \quad \forall t \in T, r \in R$$

- No team may have more than $\text{MAX_STREAK} = 2$ consecutive home or away matches.

H14 - Final-Round Simultaneous Kickoff Slot

$$\text{Sum}_{\{s \text{ in FR_SLOTS}\}} \text{fr_slots} = 1$$

$\text{Sum}_{\{v\}} x[m, s, v] = \text{fr_slots}$, for all $m \text{ in } M$: $\text{round}(m) = 34$, s
 in FR_SLOTS

$$\sum_{s \in \text{FR_SLOTS}} \text{fr_slots}_s = 1$$

$$\sum_v x_{m,s,v} = \text{fr_slots}_s, \quad \forall m \in M: \text{round}(m) = 34, \forall s \in \text{FR_SLOTS}$$

- All Round 34 matches must share exactly one kickoff slot s

H15 — Final-Round Single Calendar Date

$\text{Sum}_{\{m: \text{round}(m) = 34, v\}} x[m, s, v] \leq \text{MATCHES_PER_ROUND} \cdot \text{fr_slots}$,
 for all $s \text{ in FR_SLOTS}$

$$\sum_{m: \text{round}(m) = 34, v} x_{m,s,v} \leq \text{MATCHES_PER_ROUND} \cdot \text{fr_slots}_s, \quad \forall s \in \text{FR_SLOTS}$$

- Because of H14, all Round 34 matches also share one calendar date. The daily cap is relaxed for that date

3.9 Objective Function

Minimize the weighted penalty sum:

$$Z = Z1 + Z2 + Z3 + Z4 + Z5 + Z6 + Z7 + Z8$$

Z1 - Round placement deviation:

$$Z1 = W_ROUND_ORDER * \sum_m \sum_{s \in \text{feasible_domain}(m)} | \text{week}(s) - \text{nominal_week}(\text{round}(m)) | * x[m, s]$$

$$Z1 = W_ROUND_ORDER \cdot \sum_{m \in M} \sum_{s \in \text{feasible_domain}(m)} | \text{week}(s) - \text{nominal_week}(\text{round}(m)) | \cdot x_{ms}$$

Z2 - Weekly underload penalty:

$$Z2 = W_WEEK_UNDERLOAD * \sum_{w \in W} \text{under_load}_w$$

$$Z2 = W_WEEK_UNDERLOAD \cdot \sum_{u \in W} \text{under_load}_w$$

Z3 - Weekly overload penalty:

$$Z3 = W_WEEK_OVERLOAD * \sum_{w \in W} \text{over_load}_w$$

$$Z3 = W_WEEK_OVERLOAD \cdot \sum_{u \in W} \text{over_load}_w$$

Z4 - Travel distance:

$$Z4 = W_TRAVEL * \sum_m \sum_{s \in \text{feasible_domain}(m)} \text{travel_km}(m) * x[m, s]$$

$$Z4 = W_TRAVEL \cdot \sum_{m \in M} \sum_{s \in \text{feasible_domain}(m)} \text{travel_km}(m) \cdot x_{ms}$$

Z5 - Tier mismatch:

$$Z5 = W_TIER_MISMATCH * \sum_m \sum_{s \in \text{feasible_domain}(m)} | \text{tier_match}(m) - \text{tier_slot}(s) | * x[m, s]$$

$$Z_5 = W_TIER_MISMATCH \cdot \sum_{m \in M} \sum_{s \in \text{feasible_domain}(m)} | \text{tier_match}(m) - \text{tier_slot}(s) | \cdot x_{ms}$$

Z6 - Evening preference:

$$Z_6 = W_EVENING_PREFERENCE * \sum_{m \in M} \sum_{s,v} \max(0, 21 - \text{kickoff_hour}(s)) * x[m,s,v]$$

$$Z_5 = W_EVENING_PREFERENCE \cdot \sum_{m \in M} \sum_{s,v} \max(0, 21 - \text{kickoff_hour}(s)) \cdot x_{m,s,v}$$

Z7 - Slot-spread penalty:

$$Z_7 = W_SLOT_SPREAD * \sum_{s \in S} \text{slot_collisions}$$

$$Z_7 = W_SLOT_SPREAD \cdot \sum_{s \in S} \text{slot_collision}_s$$

Z8 - Non-primary venue penalty:

$$Z_8 = W_STADIUM_MAINTENANCE_OVERLAP * \sum_{(v,d)} \text{overlap}[v,d]$$

$$Z_8 = W_STADIUM_MAINTENANCE_OVERLAP \cdot \sum_{(v,d)} \text{overlap}_{v,d}$$

3.9.1 Weekly load auxiliary definitions

$$\text{load_w} = \sum_m \sum_{s: \text{week}(s)=w} x[m,s], \text{ for all } w \text{ in } W$$

$$\text{load_w} = \sum_{m \in M} \sum_{s: \text{week}(s)=w} x_{ms} \quad \forall w \in W$$

The deviation variables are linearisations of the $\max(0, \cdot)$ operator:

$$\text{under_load_w} \geq \text{SOFT_MIN_WEEK} - \text{load_w}, \text{ under_load_w} \geq 0, \quad \text{for all } w \text{ in } W$$

$$\text{under_load_w} \geq \text{SOFT_MIN_WEEK} - \text{load_w} \quad \forall w \in W$$

$$\text{under_load_w} \geq 0 \quad \forall w \in W$$

$$\text{over_load_w} \geq \text{load_w} - \text{SOFT_MAX_WEEK}, \text{ over_load_w} \geq 0, \quad \text{for all } w \text{ in } W$$

$$\text{over_load_w} \geq \text{load_w} - \text{SOFT_MAX_WEEK} \quad \forall w \in W$$

$$\text{over_load_w} \geq 0 \quad \forall w \in W$$

Since the objective minimises these variables, the solver sets:

$$\text{under_load_w} = \max(0, \text{SOFT_MIN_WEEK} - \text{load_w})$$

$$\text{over_load_w} = \max(0, \text{load_w} - \text{SOFT_MAX_WEEK})$$

3.10 Objective Weights

Weight	Value	Meaning
W_ROUND_ORDER	100	Penalty per week deviation from the nominal round week. Keeps rounds in expected calendar positions.
W_WEEK_UNDERLOAD	50	Penalty per match below SOFT_MIN_WEEK in any calendar week.
W_WEEK_OVERLOAD	50	Penalty per match above SOFT_MAX_WEEK in any calendar week.
W_TRAVEL	1	Penalty per km of away-team travel. Low weight; acts mainly as a tie-breaker.
W_TIER_MISMATCH	20	Penalty per tier level difference between match quality and slot quality.
W_EVENING_PREFERENCE	50	Per-hour penalty for kickoff before 21:00.
W_SLOT_SPREAD	500	Penalty per slot with more than one assigned match.
OVERLAP_PENALTY	500	Penalty per stadium service-gap violation.

Table 14: Objective Weights

3.11 Model Summary

3.11.1 Hard constraints

ID	Short description	Notes
H1	Sum $x[m,s,v] = 1$	Assignment completeness
H2	Derby $\Rightarrow$ tier-1 slot	is_derby flag
H3	Team x date ≤ 1	No double-booking per team per day

ID	Short description	Notes
H4	Venue x slot ≤ 1	No stadium double-booking
H5	Matches per day ≤ 3	Daily match cap
H6	Matches per slot ≤ 2	Slot concurrency cap
H7	4-day sliding window ≤ 1 per team	3 full rest days minimum
H8	match_time and match_date definitions	Linear-combination auxiliary vars
H9	round_min / round_max	Round-level time bounds
H10	$\text{round_max}[r] + \text{MIN_ROUND_GAP} \leq \text{round_min}[r+1]$	Strict chronological round ordering
H11	Weekly load ≤ 18	Hard weekly cap
H12	CAF buffer exclusion	Bidirectional 4-day exclusion zone
H13	Streak ≤ 2	Enforced in fixture pre-solve
H14	Round 34 shared kickoff slot	All 9 final-round matches in one slot
H15	Round 34 single date	Daily cap relaxed to 9 for that date

,Table 15: Hard Constraints

3.11.2 Objective terms

Term	Description	Weight
Z1	Round placement	100
Z2	Weekly underload	50
Z3	Weekly overload	50
Z4	Travel distance	1
Z5	Tier mismatch	20

Table 16: Objectives

CHAPTER 4

SYSTEM DESIGN

4 System Architecture and Component Overview

The system is a single-process desktop/web application built on Streamlit. There is no client-server split, no database server, and no network API; everything – the UI, the solver, and the file outputs – runs inside the same Python process on the user's machine (or on Streamlit Community Cloud).

4.1 Layers

- **Presentation layer** – `streamlit_app.py`. Renders the sidebar controls, the five main tabs (Explore, Run & progress, Insights, Artifacts, Browse files), and reads back whatever CSV/JSON artifacts exist in `output/` to populate charts and tables.
- **Orchestration layer** – `main.py` for the command-line path and the `run_clicked` block inside `streamlit_app.py` for the UI path. Both call the same underlying phase functions in the same order; the Streamlit version simply wraps each phase call in a worker thread so the progress indicators in the UI can update live.
- **Domain / solver layer** – `src/data_loader.py`, `src/fixture_generator.py`, `src/slot_domain.py`, `src/baseline_solver.py`, `src/baseline_retry.py`, `src/output_writer.py`, `src/validation.py`. This is where the actual scheduling logic lives, built around Google OR-Tools' CP-SAT constraint solver.
- **Storage layer** – plain files. Two Excel workbooks (`Data_Model.xlsx`, `expanded_calendar.xlsx`) are the inputs; everything the pipeline produces is written back out as CSV/JSON files under `output/` and `output/phases/`. There is no relational database anywhere in the system.

4.2 Component Diagram

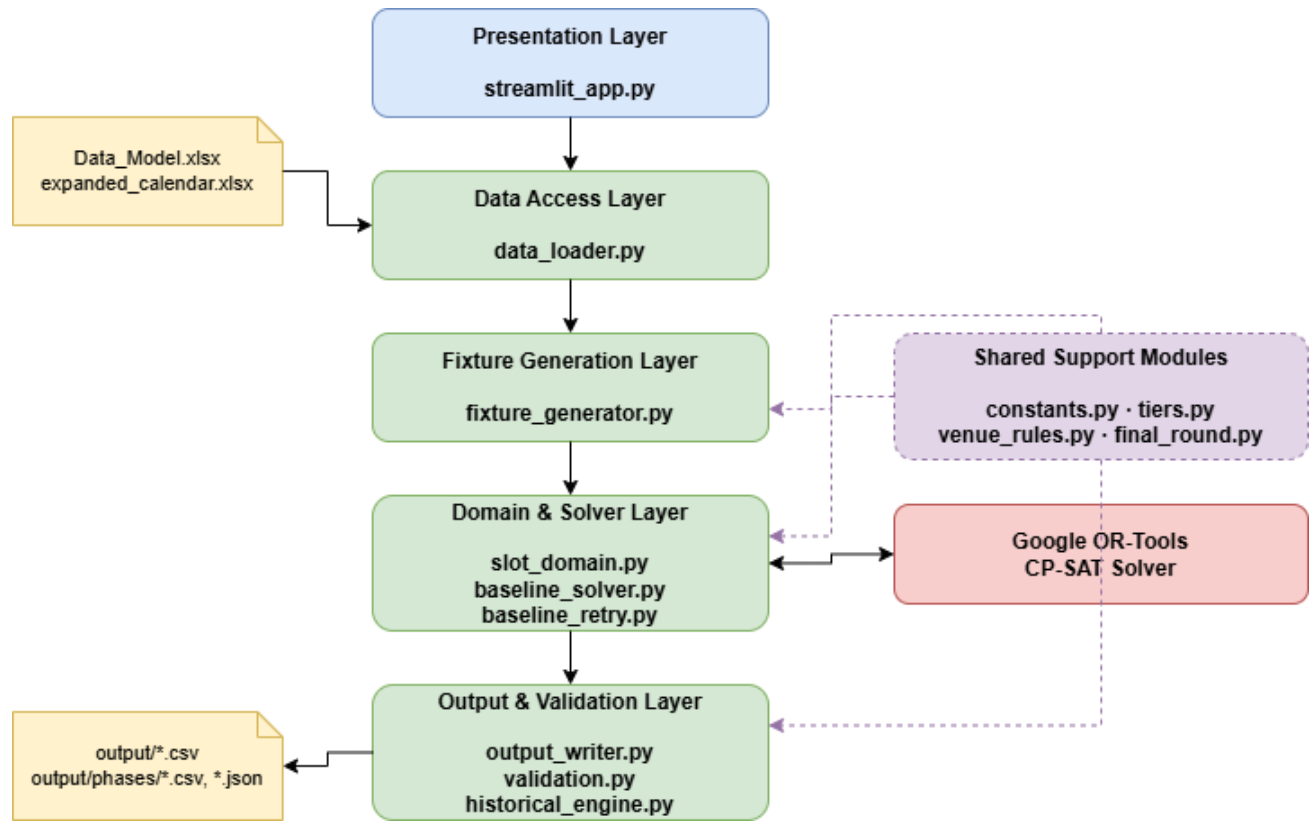


Figure 3 : System Component Diagram

4.3 Deployment Architecture

The system is designed to run on a single machine (local or cloud). The Streamlit server is started with Streamlit run streamlit_app.py and serves the browser UI over HTTP on localhost:8501 (or a cloud-assigned port). The CP-SAT solver (Google OR-Tools) runs in-process and uses the NUM_WORKERS constant to control the number of CPU threads it spawns. Monte Carlo batch runs use Python's ProcessPoolExecutor and the --parallel flag to utilise additional cores.

4.4 Class / Module Diagrams

The codebase favours small data classes and free functions over deep class hierarchies, which fits a constraint-programming pipeline where each phase is really a pure transformation: data in, a new structure out. The diagram below should be read as a module/data-class diagram rather than a classical object-oriented class diagram with inheritance, since there is almost no inheritance in the project.

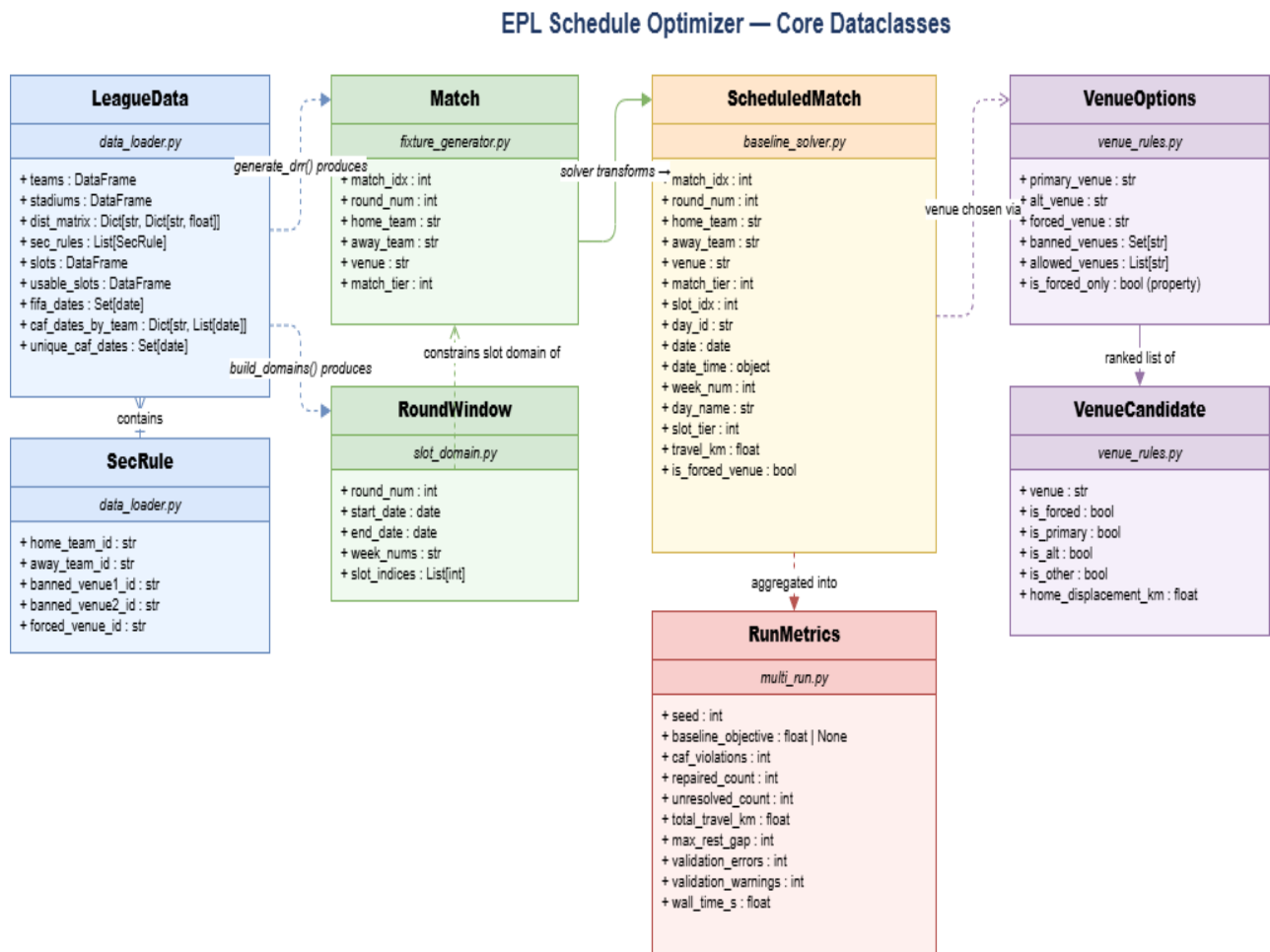


Figure 4 : Class Diagram

4.4.1 Key Data Structures Used by the Final Pipeline

- **LeagueData (src/data_loader.py)**

The single object returned by `load_data()`. Holds everything the rest of the pipeline needs in memory: `teams`, `stadiums`, `dist_matrix`, `sec_rules`, `slots`, `usable_slots`, `fifa_dates`, `caf_blockers`, `caf_dates_by_team`, `unique_caf_dates`.

- **Match (src/fixture_generator.py)**

One row of the fixture list before scheduling: `match_idx`, `round_num`, `home_team`, `away_team`, `venue` (the team's default home stadium unless a security rule forces another venue), and `match_tier` (derived from the two teams' league tiers).

- **RoundWindow (src/slot_domain.py)**

A data class describing the calendar window allowed for one abstract round (start date, end date, the calendar weeks it touches, and the list of usable-slot row indices inside that window). The CAF-aware filtering is applied at this level.

- **ScheduledMatch (src/baseline_solver.py)**

The output of the solver: a `Match` enriched with the concrete slot it was assigned (date, date-time, weekday, calendar week, slot tier), the venue used, the travel distance for the away team, and a flag for whether the venue was a forced security-rule venue. This is the structure that flows into `output_writer.py` and `validation.py` and is ultimately what the user sees in the Explore and Insights tabs.

- **RunMetrics (src/multi_run.py)**

Used only by the batch/Monte-Carlo entry point. Summarizes one solver run – objective value, total travel, CAF-related counts, validation error/warning counts – so many seeds can be compared and the best one re-materialized as the final schedule.

4.5 Sequence Diagrams (Key Use Cases)

4.5.1 Use Case: Run the Full Pipeline from the Streamlit UI

This is the main use case of the application – the user opens the app, optionally tweaks a constant in the sidebar, and clicks “Run full pipeline.”

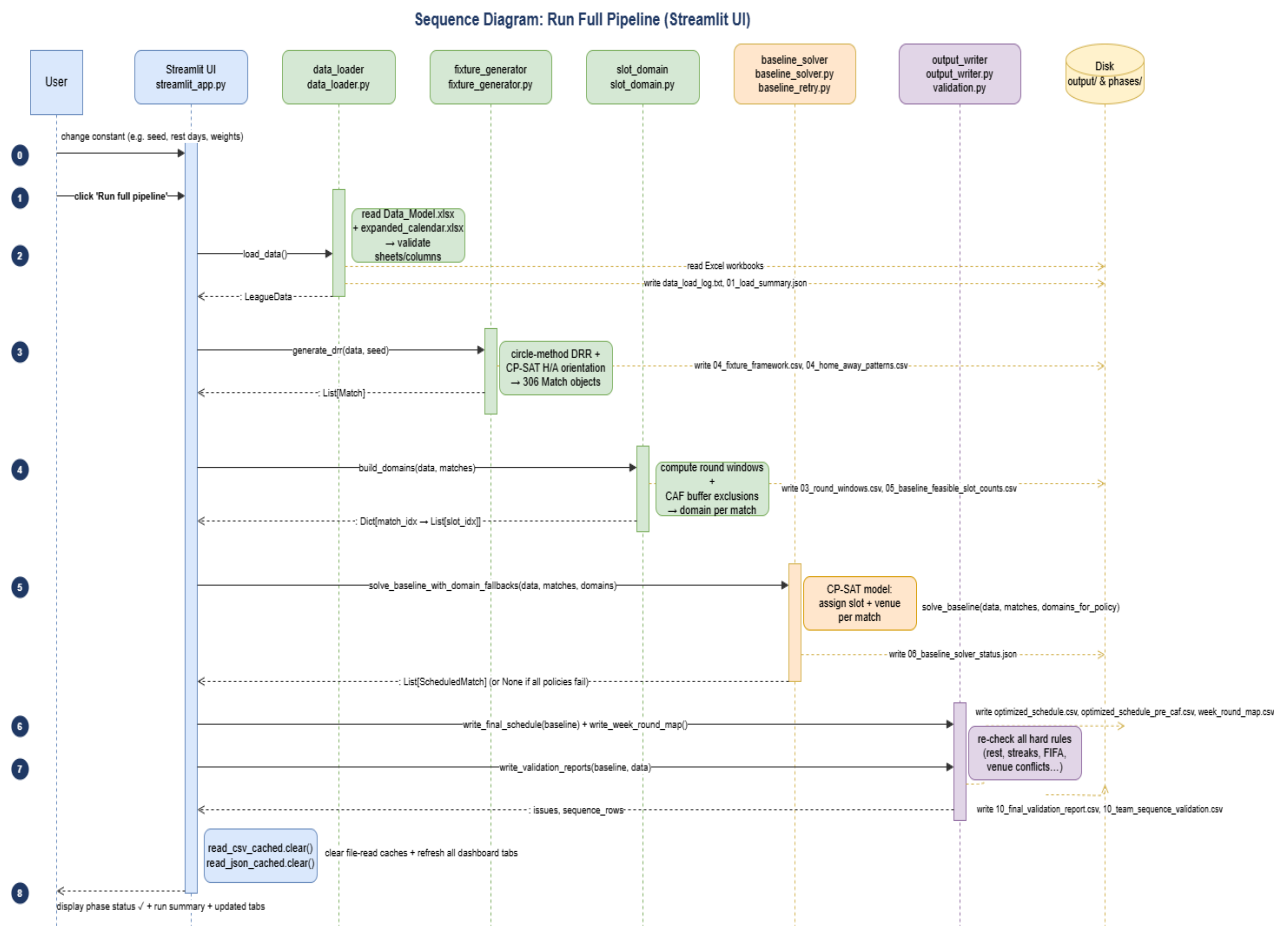


Figure 5: Full Pipeline Use Case

4.5.2 Use Case: Explore a Generated Schedule (Team Chooser)

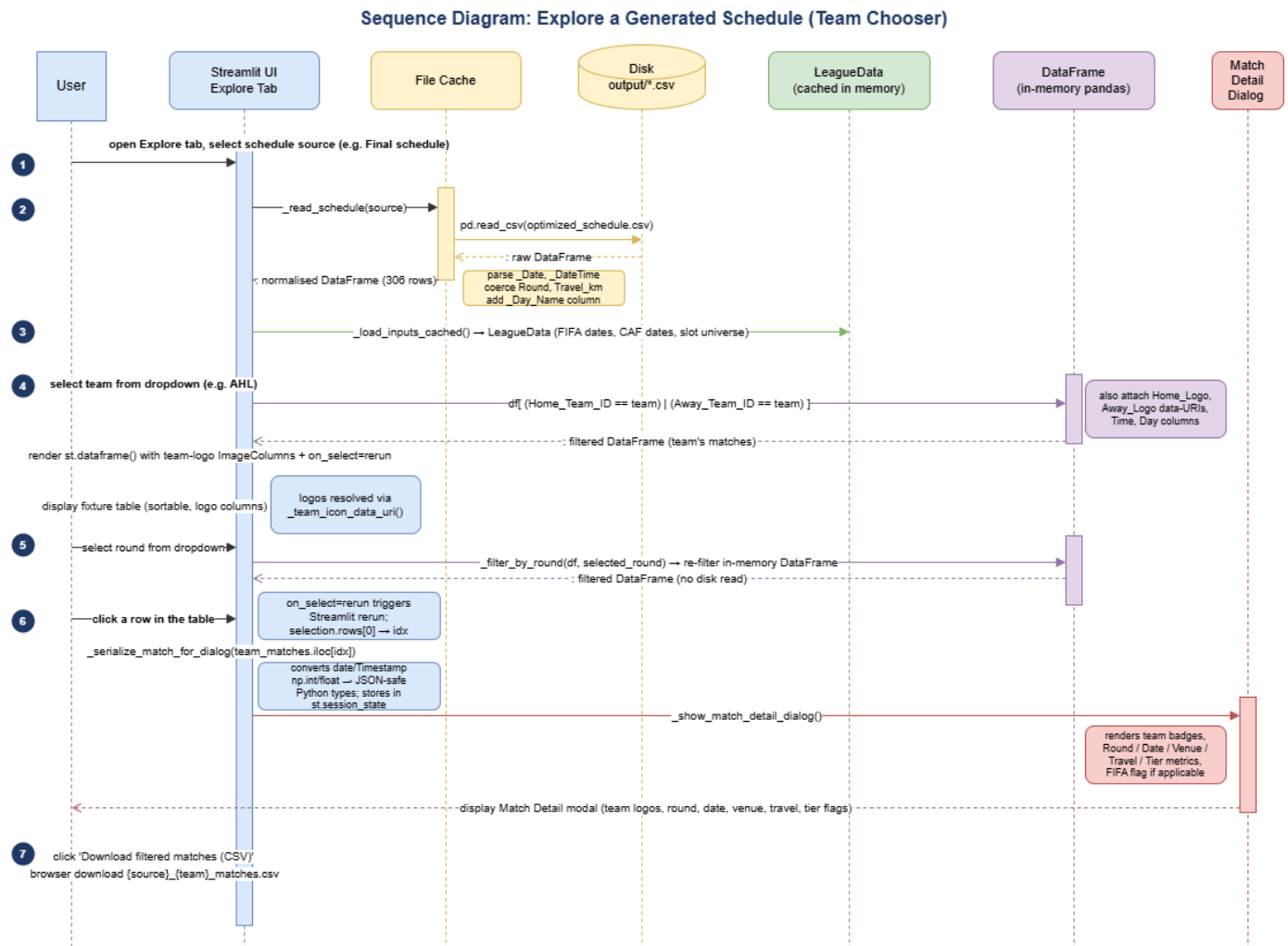
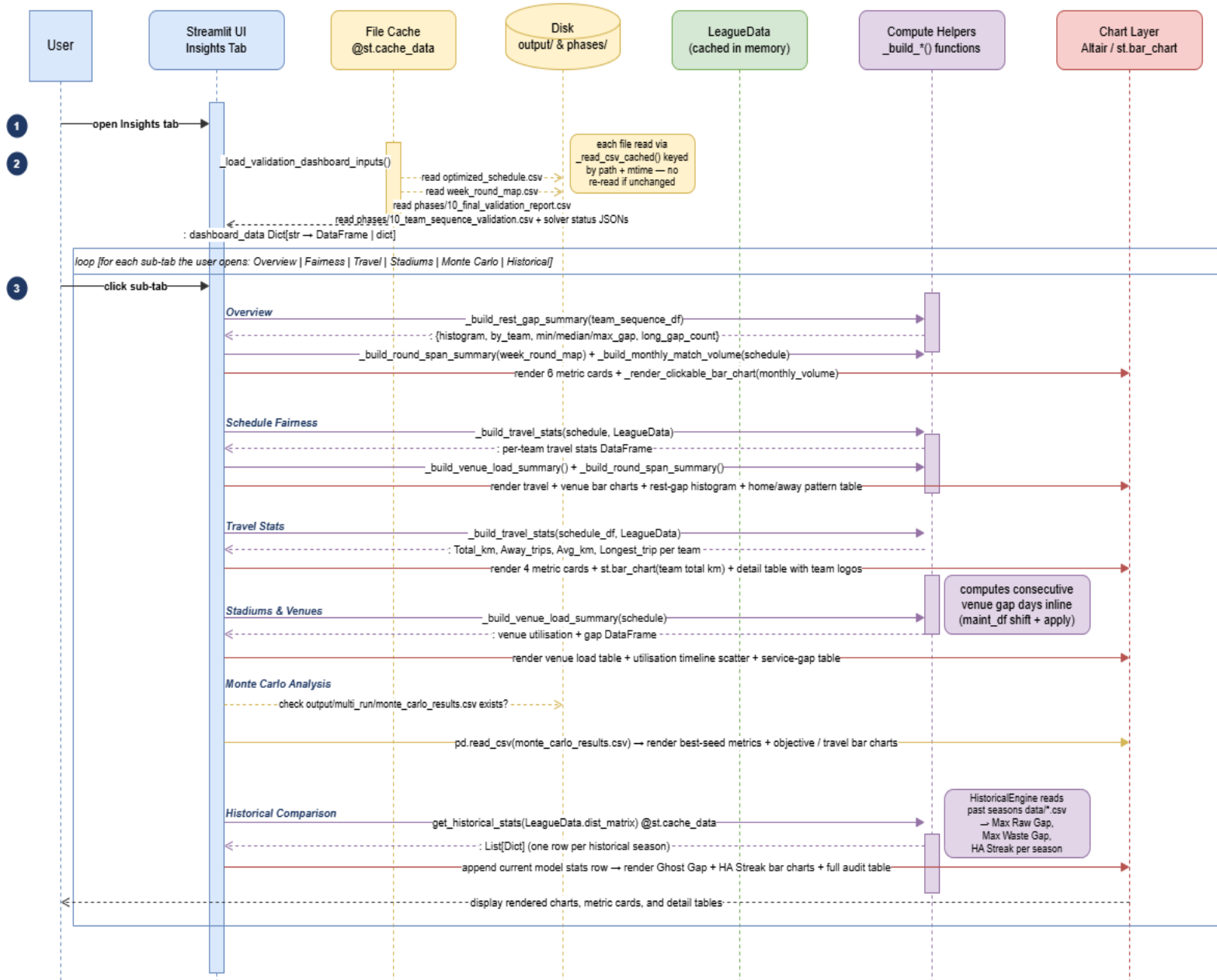


Figure 6: Generated Schedule Use Case

4.5.3 Use Case: Inspect Schedule Quality (Insights Tab)

All six sub-tabs follow the same pattern: read one or more cached CSV/JSON artifacts, summarize them in a small pandas helper function, render an Altair bar chart with a click-selection, and show the underlying rows for whatever bar the user clicked. The Historical sub-tab additionally instantiates `HistoricalEngine`, which re-parses six seasons of real Egyptian Premier League CSV data from disk on every render (cached with `@st.cache_data`) purely for the side-by-side comparison chart against the optimizer's own numbers.

Sequence Diagram: Inspect Schedule Quality (Insights Tab)



CHAPTER 5

IMPLEMENTATION

5 I Implementation Environment and Setup

5.1 Language, Runtime, and Core Libraries

The whole project is written in Python 3 and organized as a `src/package` imported by two entry points: `main.py` for command-line runs and `streamlit_app.py` for the interactive dashboard. The table below lists the third-party libraries the system depends on at runtime.

Library	Version family	Role in the project
ortools (cp_model)	OR-Tools CP-SAT	The constraint solver that assigns every match to a calendar slot and venue; also used for the small home/away-orientation model in fixture generation.
pandas / numpy	pandas 2	All tabular data handling: reading the Excel workbooks, building the schedule DataFrame, and every aggregation shown in the Insights tab.
streamlit	1.3	The web UI framework the whole dashboard is built on (tabs, widgets, caching decorators, <code>st.dialog</code> modals).
altair	5	Interactive, click-to-filter bar charts used throughout the Insights tab.
openpyxl	3	The Excel engine pandas uses under the hood to read <code>Data_Model.xlsx</code> and <code>expanded_calendar.xlsx</code> .

Table 17: Used Libraries

5.2 Project Layout

The repository is organized so that source code, input data, and generated output never mix:

```
project_root/
├── main.py          # CLI entry point
├── streamlit_app.py  # Dashboard entry point
├── src/
│   ├── constants.py    # every tunable rule and file path
│   ├── data_loader.py  # Excel ingestion + validation
│   ├── fixture_generator.py # double round-robin (DRR) generation
│   ├── slot_domain.py  # round windows + feasible-slot domains
│   ├── baseline_solver.py # CP-SAT calendar/venue assignment
│   ├── baseline_retry.py # domain-fallback retry wrapper
│   ├── output_writer.py # writes optimized_schedule.csv etc.
│   ├── validation.py   # post-solve hard-rule checks
│   ├── historical_engine.py # benchmark vs. real EPL seasons
│   ├── tiers.py, venue_rules.py, final_round.py
│   └── multi_run.py     # Monte Carlo batch runner
├── data/
│   ├── Data_Model.xlsx
│   └── expanded_calendar.xlsx
├── past seasons data/    # historical benchmark CSVs
└── output/               # created at runtime, never committed
    └── phases/
```

5.3 Running the Project

Two supported ways to run the optimizer, matching the two entry points:

- **Dashboard:** streamlit run streamlit_app.py — opens the browser UI described in Chapter 4; the user sets the seed and any constant overrides in the sidebar and clicks “Run full pipeline.”
- **Command line / batch:** python main.py --seed 88 --runs 1 for a single deterministic run, or --runs N (with --parallel workers) to launch a Monte Carlo sweep across N seeds via

multi_run.py, check pointing progress to output/multi_run/monte_carlo_results.csv after every completed seed.

Both entry points expect Data_Model.xlsx and expanded_calendar.xlsx to already exist at the paths declared in constants.py (DATA_MODEL_PATH, EXPANDED_CALENDAR_PATH). There is no installer or setup wizard; a working Python environment with the libraries in Table 5.1 installed is the only prerequisite.

5.4 Key Implementation Details

5.4.1 Fixture Generation: Circle Method + CP-SAT Orientation

The 306-match double round-robin is built in two stages inside fixture_generator.py. First, the classic circle method produces 17 rounds of unordered pairings for a shuffled team list (one team fixed, the other 16 rotating around it each round) this guarantees every team meets every other team exactly once in the first leg. Second, because the circle method does not decide who is home and who is away, a small CP-SAT model (_solve_orientation) assigns a home/away orientation to every first-leg pairing so that, across the full 34-round season, no team exceeds a 3-window / 5-window home-away balance (no more than 2 home or away games in any 3 consecutive rounds, and a 2–3 split in any 5-round rolling window). The second leg simply mirrors the first leg with home and away swapped.

The orientation solve is attempted with three progressively looser rule sets (orient_pairings_with_valid_patterns): first with the rolling-5 balance rule and balanced season edges, then without balanced edges, and finally without the rolling-5 rule at all if the first two attempts fail within their 15-second time budget. This mirrors the same “try strict, fall back to looser” philosophy used later in the baseline solver.

5.4.2 Slot Domains and the CAF Rest-Day Rule

Before the main solve, slot_domain.py builds 34 chronological round windows from the usable (non-FIFA) calendar slots, then computes, for every match, the specific list of slot indices it is allowed to use — its domain. This is also where the CAF rest-day rule is enforced. The _build_blocked_by_team pre-computes, for every CAF-participating team, the set of calendar slots that fall within MIN_REST_DAYS_CAF + 1 days of one of that

team's continental fixtures, and those slots are simply excluded from the match's domain before the solver ever runs. Because the CP-SAT decision variable for a match/slot pair is only created for slots inside that match's domain, a CAF violation is not something the solver has to be told to avoid — it is a value the variable cannot take in the first place. If excluding every CAF-blocked slot would leave a match with zero playable slots in its round window, the buffer is relaxed for that single match rather than making the whole round infeasible, and the match is flagged in `05_baseline_feasible_slot_counts.csv` (`caf_filter_relaxed_for_repair` column) purely for diagnostic visibility.

Round windows themselves are also adaptive. If a round's compact 5-day window has too few feasible slots for its 9 matches, `_expand_non_final_round_windows` grows that specific round's window one day at a time (never touching the strict monotonic ordering between rounds) until either the slot count and per-match feasibility thresholds are met or the window hits its policy-defined ceiling.

5.4.3 The Baseline CP-SAT Model

`baseline_solver.py` builds one CP-SAT model per run. Each match/slot/venue combination inside the match's domain becomes a boolean decision variable; exactly one must be true per match. The hard constraints layered on top are:

- **Rest days:** no two matches for the same team fall within `MIN_REST_DAYS_LOCAL` days of each other (checked with a sliding date window per team, not just consecutive matches).
- **Home/away and daily/weekly capacity:** at most `MAX_MATCHES_PER_DAY` matches share a calendar date, at most `MAX_MATCHES_PER_SLOT` share a kickoff slot, and weekly load stays under `HARD_MAX_MATCHES_PER_WEEK`.
- **Round ordering:** the latest match of round `N` must fall at least `MIN_DAYS_BETWEEN_ROUNDS` days before the earliest match of round `N+1`, modeled with auxiliary per-round min/max day variables
- **Venue conflicts:** no venue hosts two matches in the same slot, and (when `MIN_STADIUM_SERVICE_GAP_DAYS > 0`) back-to-back use of the same

non-forced venue within the service-gap window is penalized rather than forbidden outright, via an overlap indicator variable added to the objective.

- **Tier-1 derbies:** any match between two Tier-1 clubs must land in a Tier-1 (weekend-evening) kickoff slot, enforced as a hard sum-equals-one constraint over that match's Tier-1 slot variables.
- **Final-round single-slot rule:** every Round 34 match is forced onto one shared calendar date and kickoff slot across the whole league, implemented by aggregating each match's per-slot indicator variables into one shared boolean per candidate slot and requiring exactly one candidate slot to be chosen league-wide.

On top of these hard constraints, a single weighted linear objective is minimized — round-order drift, week under/overload, travel distance, match-tier vs. slot-tier mismatch, evening-kickoff preference, slot-spread (discouraging clustering multiple matches in one kickoff slot on the same day), and the venue-relief penalties described above. All weights live in `constants.py` (`W_ROUND_ORDER`, `W_TRAVEL`, `W_TIER_MISMATCH`, and so on) and are exposed as sliders in the Streamlit sidebar.

5.4.4 Domain-Fallback Retries and the Final-Round Rescue Model

A model this constrained can be infeasible under its strict round-window policy.

`baseline_retry.py` wraps the solve in a three-attempt loop — compact windows, then extended “EPL-style” spillover windows, then full-season spillover tails — rebuilding domains and re-solving each time until one attempt returns a schedule or all three are exhausted.

A second, more targeted fallback exists specifically for Round 34. Because the final round is forced onto a single shared slot league-wide, it is disproportionately likely to be the reason a solve fails even when Rounds 1–33 are easy. When the strict combined model is infeasible (or its final-round shared-slot constraint has no viable candidate slot at all), the solver instead solves Rounds 1–33 alone first, then builds a dedicated, much smaller CP-SAT model that fixes those 297 matches and searches only for a common slot and venue set for the remaining 9 final-round matches. In this rescue model, rules that were hard constraints in the main solve (rest-day buffers, tier-1 slot requirement, round-

gap spacing, weekly caps, forced-venue rules) become penalized soft violations instead, each with its own weight (`FINAL_ROUND_RESCUE_*_PENALTY` constants), so the model can still return a usable, ranked-best schedule even in a genuinely tight calendar rather than failing outright.

5.4.5 Post-Solve Validation

Once a schedule is produced, `validation.py` re-checks every hard rule end-to-end — rest gaps, home/away streaks, round ordering, FIFA-date exclusion, CAF buffers, venue/slot conflicts, and the final-round rule — and writes a pass/fail report (`10_final_validation_report.csv`) plus a per-team match sequence (`10_team_sequence_validation.csv`) that the Insights tab reads directly. This stage is deliberately read-only: unlike the CAF-repair pass from an earlier version of the project, it does not move any match. Its only job is to give an honest, independent confirmation that the constraints the solver was asked to satisfy were in fact satisfied, which also makes it useful as a regression check whenever a constant in `constants.py` changes.

5.4.6 Streamlit Application: Caching and Non-Blocking Runs

Two implementation problems are specific to wrapping a multi-minute CP-SAT solve inside a reactive web UI: keeping the page responsive while the solver runs, and avoiding repeated, expensive file reads every time the user clicks something unrelated (Streamlit reruns the whole script on every interaction).

For responsiveness, `_run_phase` in `streamlit_app.py` runs each pipeline phase on a background thread and polls it from the main thread with `thread.join(timeout=0.2)`, so Streamlit keeps servicing its websocket connection instead of freezing for the full duration of a phase. Standard output and standard error are captured with `redirect_stdout / redirect_stderr` into an in-memory buffer, which the main thread tails into the “Live stdout” panel only when the “Show live logs” toggle is on.

For caching, two layers are used: `st.cache_resource` around `load_data()` so the (relatively expensive) Excel parse only happens once per session, and `st.cache_data` around every CSV/JSON reader, keyed on a (path, modification-time) tuple rather than the path alone — so a file that has not changed since the last read is served from cache, but a freshly re-run pipeline invalidates the cache automatically because its output files get a new mtime.

After a successful run, the app explicitly calls `_read_csv_cached.clear()` and `_read_json_cached.clear()` so the Explore and Insights tabs are guaranteed to reflect the new files on the very next rerun rather than waiting for an mtime check that could, in principle, race the filesystem.

5.5 Sample Screenshots / Output Samples

The screenshots below are captures of the running Streamlit application; the output sample that follows them is drawn directly from the real column layout the pipeline writes.

Run configuration

DRR seed

88

Data model path

data/Data_Model.xlsx

Expanded calendar path

data/expanded_calendar.xlsx

Model variables

Applied to the next pipeline run in this Streamlit session.

☐ Show live logs

Rest and streak rules

Min local rest days

3

Min CAF rest days

3

Preferred CAF rest days

5

Max consecutive home

2

Max consecutive away

2

capacity

Objective weights

Solver limits

Baseline solver limit (sec)

600

Repair solver limit (sec)

60

UI & Performance (Cloud Safety)

Figure 8: Sidebar

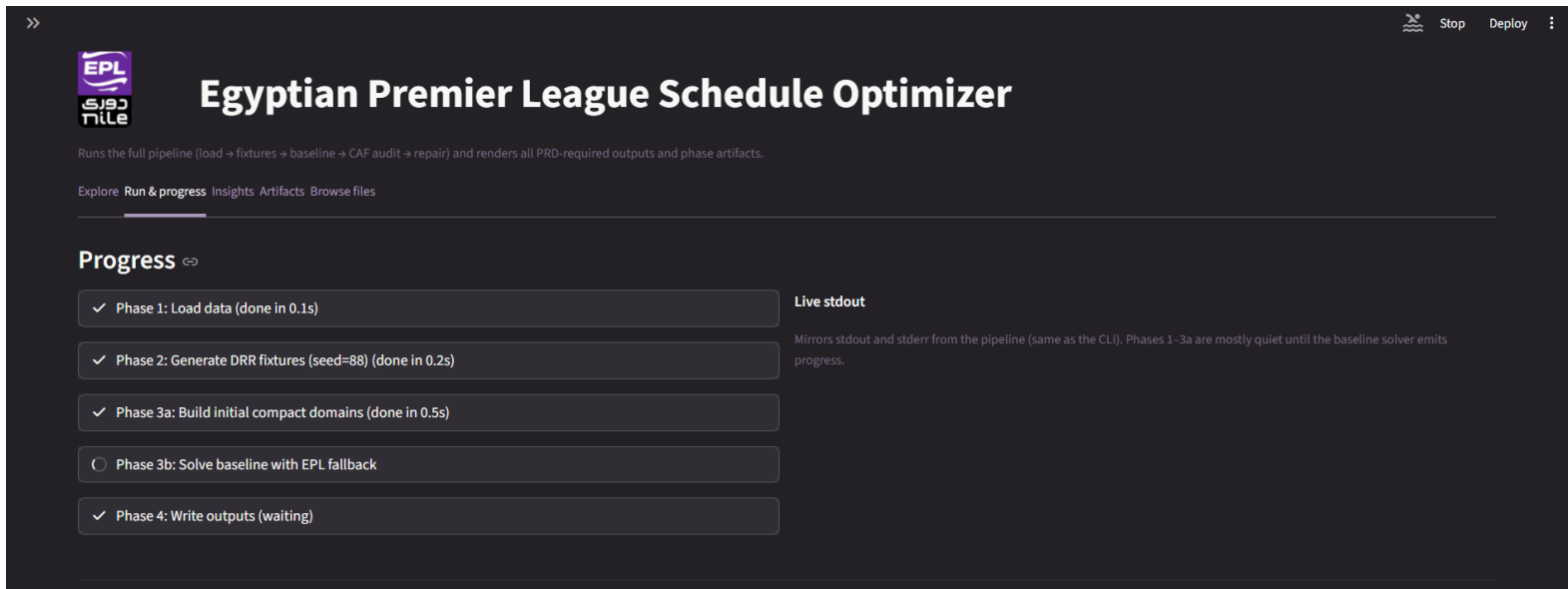


Figure 9: “Run & progress” tab mid-run

Team chooser Head to Head Calendar

Team chooser

Pick a team

AHL — Al Ahly SC

AHL — Al Ahly SC

☒ Include home ☒ Include away

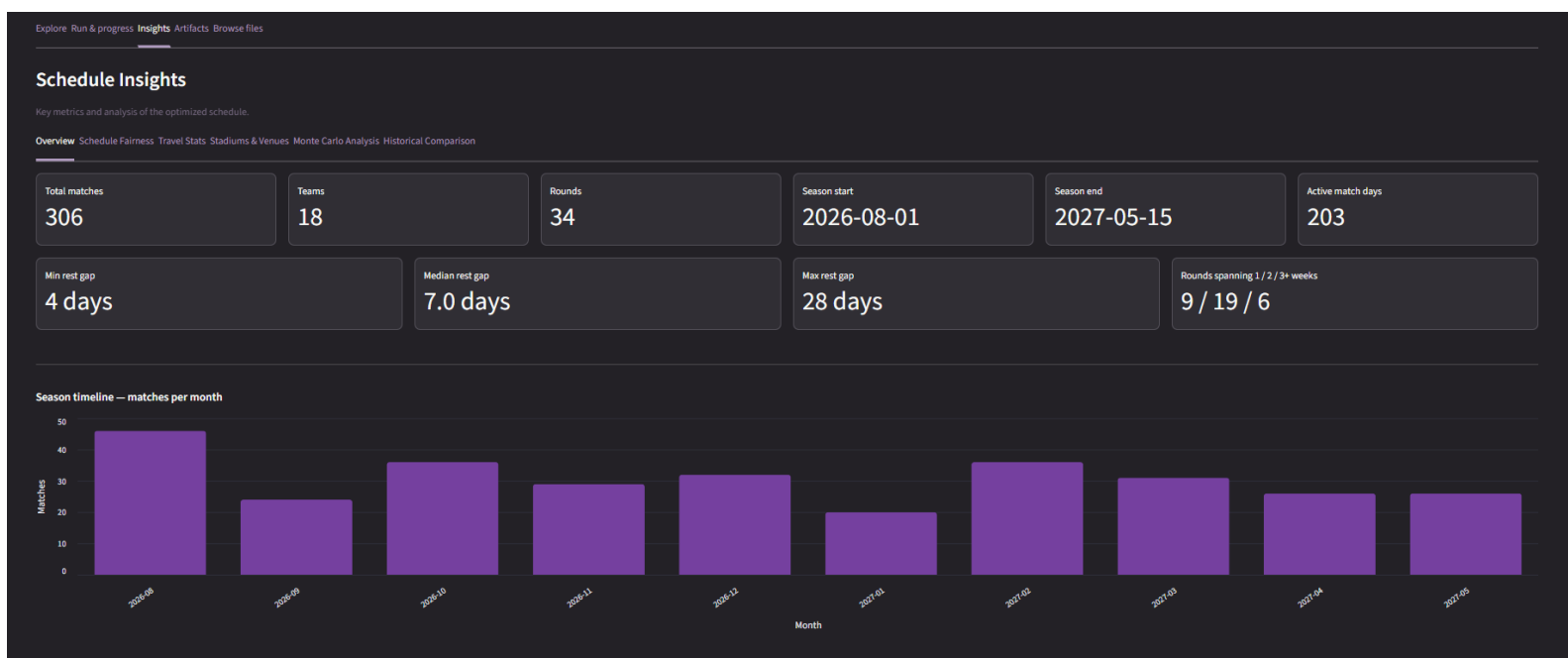
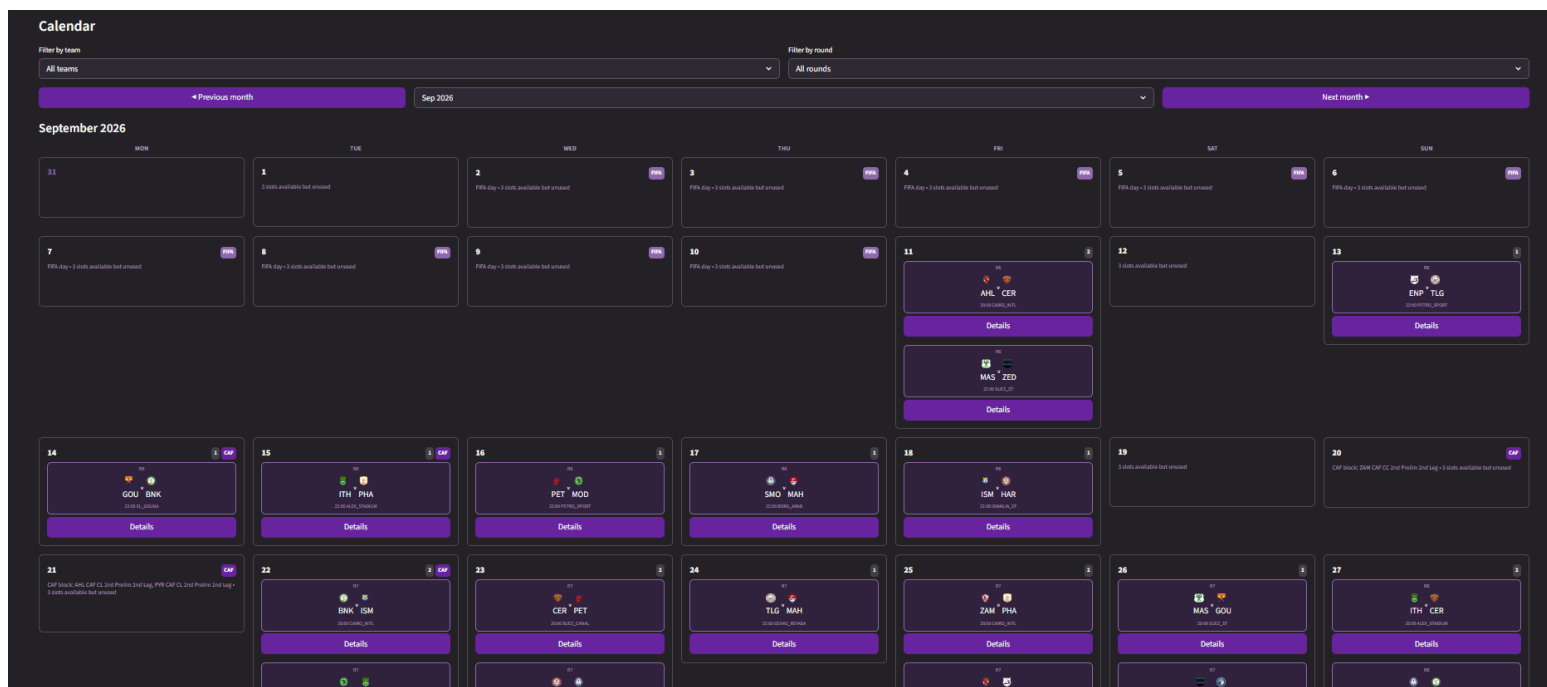
Sort by: Date Round: All rounds

Matches for team **AHL** : 34

Click a row to view full match details.

Round	Date	Day	Time		Home		Away	Venue	Slot_tier
☐	1 2026-08-02	Sunday	20:00		AHL		BNK	CAIRO_INTL	2
☐	2 2026-08-09	Sunday	22:00		MAS		AHL	BORG_ARAB	2
☐	3 2026-08-14	Friday	20:00		AHL		PYR	CAIRO_INTL	1
☐	4 2026-08-19	Wednesday	22:00		PHA		AHL	BORG_ARAB	2
☐	5 2026-08-26	Wednesday	22:00		MOD		AHL	AL_SALAM	2
☐	6 2026-09-11	Friday	20:00		AHL		CER	CAIRO_INTL	1
☐	7 2026-09-25	Friday	22:00		AHL		ENP	AL_SALAM	1
☐	8 2026-09-29	Tuesday	20:00		PET		AHL	PETRO_SPORT	2
☐	9 2026-10-03	Saturday	20:00		ITH		AHL	ALEX_STADIUM	1
☐	10 2026-10-16	Friday	22:00		AHL		ZAM	CAIRO_INTL	1
☐	11 2026-10-26	Monday	20:00		ZED		AHL	CAIRO_INTL	2

Figure 10: “Explore” tab — Team chooser



Match Details



AHL

AHL (Home)

VS



ZAM

ZAM (Away)

Round

10

Kickoff

22:00

Venue

CAIRO_INTL

Date

2026-10-16

Day

Friday

Away Travel

0.0 km

Slot Tier

1

Match Tier

1

Home Tier

1

Away Tier

1

Figure 13: Match Details Card

5.6 Final Schedule Output (optimized_schedule.csv)

output_writer.py writes following columns for every scheduled match, sorted by date and then round. This is the file all tabs and download buttons in the Explore/Artifacts tabs reads from.

Column	Example value	Column
Round	17	Home_Team_ID
Calendar_Week_Num	23	Away_Team_ID
Day_ID	D_20260218	Venue_Stadium_ID
Date	2026-02-18	Travel_km
Date_time	2026-02-18 20:00	Slot_tier
Match_Tier	1	Is_FIFA / Is_CAF
Postponed	False	Postponement_Status / Reason

Figure 14: Output Format

CHAPTER 6

TESTING AND EVALUATION

6 Testing

6.1 Testing strategy

No separate pytest suite was written for this project. Instead, correctness was verified through four complementary mechanisms that are all part of the shipped pipeline itself, so every run of the application is, in effect, also a test run.

6.1.1 Build-Time Structural Assertions

`fixture_generator.py` calls `_validate_drr` immediately after generating the double round-robin, before the matches are ever handed to the solver. This function contains a series of plain Python assert statements that check the fixture list is structurally sound: exactly 306 matches, exactly 306 distinct ordered (home, away) pairs, every team playing exactly 17 home and 17 away matches, and exactly 9 matches per round with no team appearing twice in the same round. If any of these fail, the pipeline stops immediately with a Python `AssertionError` rather than silently continuing with a broken fixture list — this is the project's first line of defense and catches fixture-generation bugs before a single CP-SAT variable is created.

6.1.2 Automated Post-Solve Validation

After every solve, `validation.py` independently re-checks the finished schedule against every hard rule from the mathematical model in Chapter 3B — it does not trust that the solver's constraints were written or wired correctly, it re-derives each rule from the raw `ScheduledMatch` list and flags any row that violates it. This acts as the project's regression test: whenever a constant in `constants.py` is changed (a rest-day rule, a weekly cap, a weight), the very next run's `10_final_validation_report.csv` shows immediately whether the change broke something. Because this check runs on every single execution, not on a separate manually triggered test command, it caught most of the edge-case bugs found during development (for example, an early version of the round-order check missed postponed matches and had to be corrected after the report surfaced repeated false positives during development).

6.1.3 Manual / Exploratory Testing via the Dashboard

The Streamlit “Artifacts” and “Browse files” tabs were used throughout development as an ad-hoc inspection tool: after any code change, a full run was triggered from the sidebar and the resulting CSVs were spot-checked directly in the browser rather than by opening the output folder in a separate file manager. This made the feedback loop between changing a constraint and seeing its effect on the schedule fast enough to be used as the primary day-to-day testing method.

6.2 Test Cases and Results

The table below lists checks validation.py performs on every run, presented as test cases

ID	Rule Tested	Method / Check Name	Severity	Result
TC-01	Fixture completeness	FIXTURE_COUNT — 306 matches, both league phases present	ERROR	PASS
TC-02	Ordered-pair completeness	ORDERED_PAIR_COUNT — every team meets every other team home and away	ERROR	PASS
TC-03	FIFA date exclusion	FIFA_DATE — no league match falls on an international break date	ERROR	PASS
TC-04	CAF rest buffer	CAF_BUFFER — CAF teams keep required days clear of continental fixtures	ERROR	PASS
TC-05	Local rest days	TEAM_REST — minimum days between any team's consecutive matches	ERROR	PASS
TC-06	Home/away streak cap	TEAM_HOME_AWAY_STREAK — no team exceeds the configured streak limit	ERROR	PASS
TC-07	Rolling-5 balance	TEAM_ROLLING5_BALANCE — 2–3 home/away split in any 5-match window	WARN	PASS (0–2 warnings typical)

ID	Rule Tested	Method / Check Name	Severity	Result
TC-08	Round chronology (team)	TEAM_ROUND_ORDER — a team's non-postponed rounds stay in order	ERROR	PASS
TC-09	Round chronology (league)	GLOBAL_ROUND_ORDER — round N finishes before round N+1 starts	ERROR	PASS
TC-10	Daily match cap	DAILY_MATCH_CAP — at most MAX_MATCHES_PER_DAY league matches share a date	ERROR	PASS
TC-11	Venue/slot conflicts	VENUE_SLOT_CONFLICT — no venue hosts two matches in one kickoff slot	ERROR	PASS
TC-12	Stadium turnaround	STADIUM_SERVICE_GAP — non-forced venue reuse respects the service gap	WARN	PASS (occasional WARN on tight calendars)
TC-13	Final-round rule	FINAL_ROUND_SINGLE_SLOT — all 9 Round-34 matches share one date/slot	ERROR	PASS
TC-14	DRR structural integrity	_validate_drr asserts (pre-solve, fixture_generator.py)	ERROR (assert)	PASS

Table 18: Test Cases

6.3 Performance Evaluation / Benchmarking

6.3.1 Solver Performance

The baseline CP-SAT model is given a hard wall-clock ceiling (BASELINE_SOLVER_TIME_LIMIT_S, 600 seconds by default) per domain-policy attempt. In practice, most seeds solve to OPTIMAL or a good FEASIBLE solution well before this ceiling under the compact round-window policy; the looser epl_relaxed and epl_full policies are only reached — and only add to total run time — on seeds where the compact policy is genuinely infeasible.

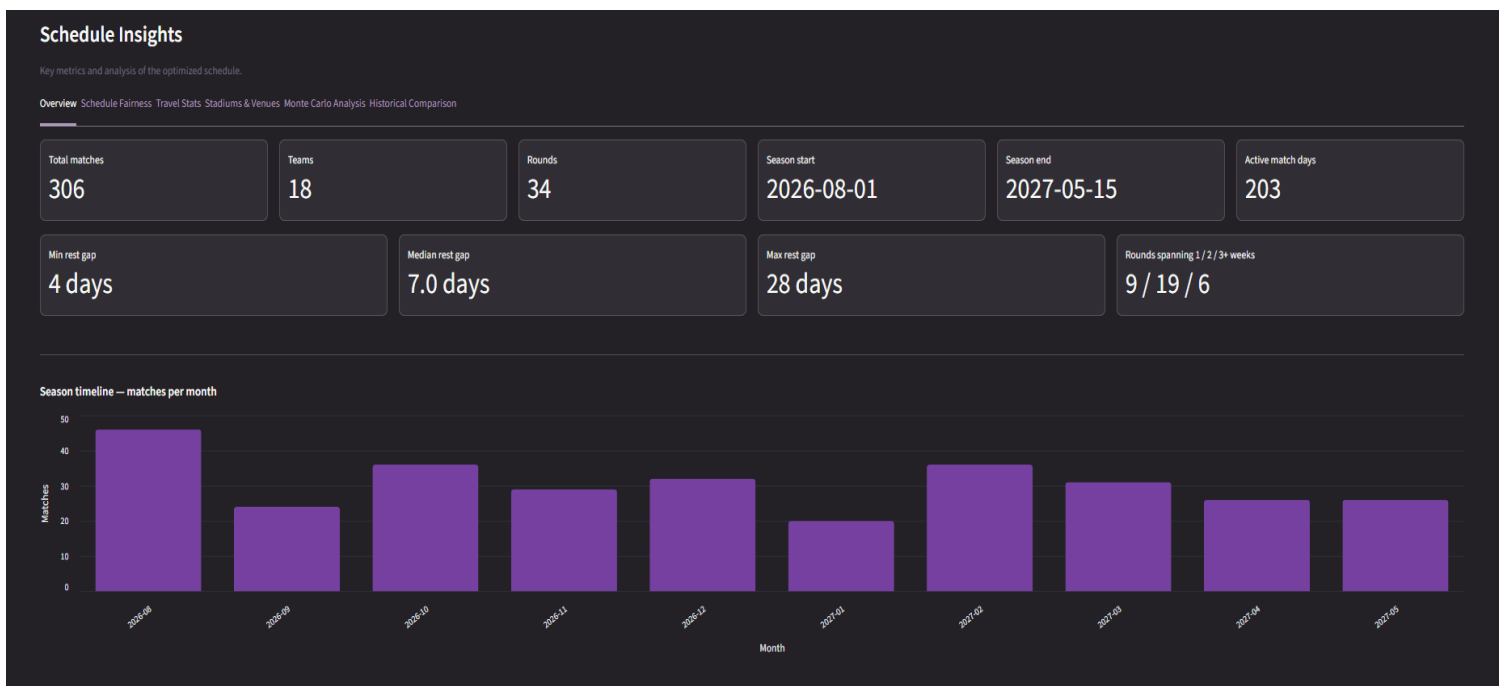


Figure 15: Results Overview

6.3.2 Evaluating Travel Efficiency

A primary objective of the optimization model is to reduce unnecessary travel fatigue for away teams, particularly for geographically distant clubs. The baseline solver achieves this through the W_TRAVEL penalty, which calculates the exact distance between the away team's home base and the assigned match venue using the origin-destination distance matrix.

By applying a per-kilometer penalty, the solver naturally favors scheduling assignments that minimize extreme cross-country travel within condensed timeframes. To validate the success of this objective, the Travel sub-tab inside the Insights dashboard aggregates the total travel distance for all 18 teams across the season

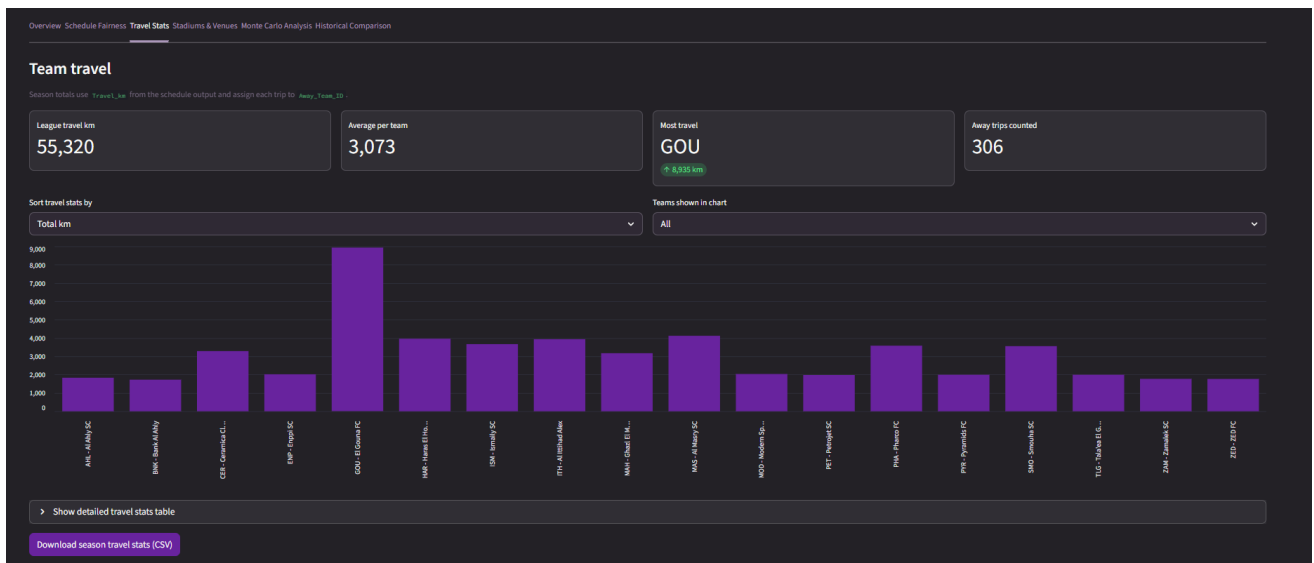


Figure 16: Travel Fairness Insights

6.3.3 Stadium Utilization and Maintenance Overlaps

The Egyptian Premier League relies heavily on shared venues. This creates a significant scheduling bottleneck. To manage this, the solver enforces a stadium service gap (MIN_STADIUM_SERVICE_GAP_DAYS), ensuring that the same pitch is not used on back-to-back days, which protects the infrastructure.

When multiple teams request the same stadium for the same calendar window, the solver evaluates the W_STADIUM_MAINTENANCE_OVERLAP penalty. The Maintenance sub-tab in the Insights dashboard visually validates this process.

Stadium Service Gaps

Configured minimum turnaround between matches at the same venue: 2 days

7 match(es) have a short turnaround at the same venue.

Venue_Stadium_ID	Round	_Date	Prev_Date	Gap_Days	Home_Team_ID	Away_Team_ID
CAIRO_INTL	17	2026-12-20	2026-12-18	2	ZAM	SMO
CAIRO_INTL	18	2026-12-22	2026-12-20	2	AHL	BNK
CAIRO_INTL	18	2026-12-24	2026-12-22	2	ZAM	TLG
CAIRO_INTL	27	2027-03-09	2027-03-07	2	BNK	MAH
CAIRO_INTL	32	2027-05-04	2027-05-03	1	AHL	TLG
CAIRO_INTL	33	2027-05-08	2027-05-07	1	AHL	MAH
CAIRO_INTL	33	2027-05-09	2027-05-08	1	ZED	GOU

Figure 17: Stadium Service Gaps

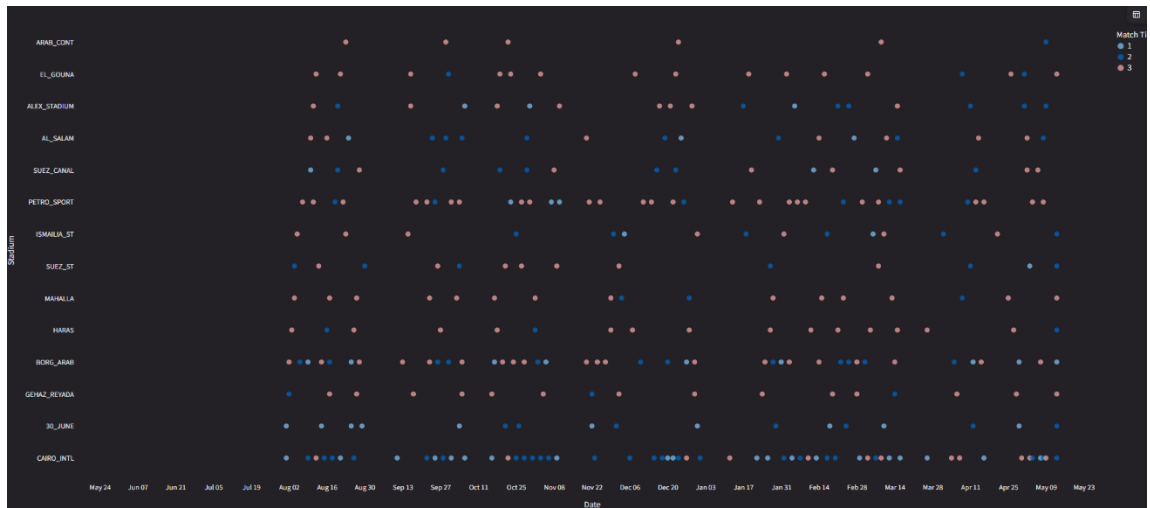


Figure 18: Venue Utilization Timeline

Matches per Venue

Venue_Stadium_ID	Matches	Share
CAIRO_INTL	57	0.1863
BORG_ARAB	41	0.134
PETRO_SPORT	35	0.1144
ALEX_STADIUM	18	0.0588
AL_SALAM	18	0.0588
30_JUNE	17	0.0556
EL_GOUNA	17	0.0556
GEHAZ_REYADA	17	0.0556
HARAS	17	0.0556
MAHALLA	17	0.0556

Figure 19: Matches per Venue:

6.3.4 Benchmarking Against Historical Seasons

The “Historical Comparison” sub-tab of Insights is the project's main built-in benchmarking tool. `historical_engine.py` re-derives, for six real Egyptian Premier League seasons, the same three headline metrics the optimizer itself is measured on: the maximum raw gap between a team's consecutive matches, the maximum ‘true waste gap’ (idle days once FIFA breaks and a CAF-match buffer are subtracted out), and the longest home/away streak any team was subjected to. The current model's own numbers are computed the same way from `optimized_schedule.csv` and appended as an extra row, so the comparison chart is built from one consistent methodology on both sides rather than mixing a strict metric for the model against a looser one for history.

This comparison is intentionally conservative in the historical direction: because the historical CSVs do not record which specific stadium a team used for a given home match, every historical fixture is assumed to be played at the team's primary stadium. The Insights tab documents this assumption directly to the user in an expandable “Methodology & Assumptions” panel .

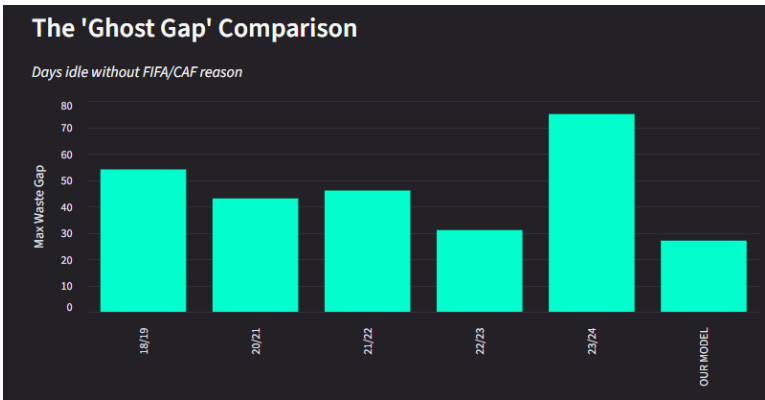


Figure 20: The 'Ghost Gap' Comparison

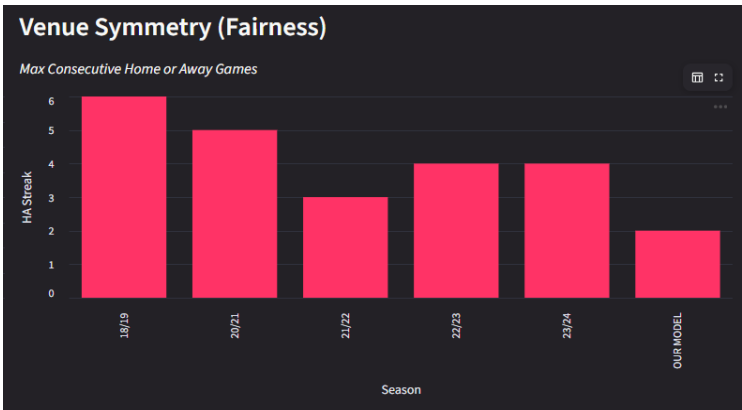


Figure 21: Venue Symmetry (Fairness)

Show full historical audit trail table

Season	Matches	FIFA Days	Max Raw Gap	Max Waste Gap	HA Streak	
18/19		306	45	63	54	6
20/21		306	25	52	43	5
21/22		306	43	58	46	3
22/23		306	53	40	31	4
23/24		306	36	91	75	4
OUR MODEL		306	45	32	27	2

📖 Methodology & Assumptions

How we compare against the past:

1. The **Primary Stadium Assumption**: Since historical CSVs don't record the specific stadium used, we assume teams played at their primary home ground (e.g., Ahly at Cairo Intl).

2. **Conservative Benchmark**: In reality, historical travel was likely *higher* than shown here because teams often moved matches to alternative stadiums.

3. The **'True Waste Gap'**: We subtract all FIFA dates and a 4-day buffer for every CAF match from the total days between games to find 'dead time' in the calendar.

💡 Insight: Your optimizer reduces the historical 'Waste Gap' from ~45 days down to 5 days, effectively saving 10 weeks of the calendar.

Figure 22: full historical audit trail table

6.3.5 Benchmarking Across Seeds (Monte Carlo)

Because a single DRR seed can, by chance, be harder or easier for the solver to schedule well, `multi_run.py` was used to sweep many seeds and rank them by the same priority order the app itself uses to pick a ‘best’ run: fewest validation errors first, then fewest unresolved matches, then lowest solver objective, then lowest total travel. The Monte Carlo sub-tab plots the resulting objective-value and travel-km distributions across the swept seeds.

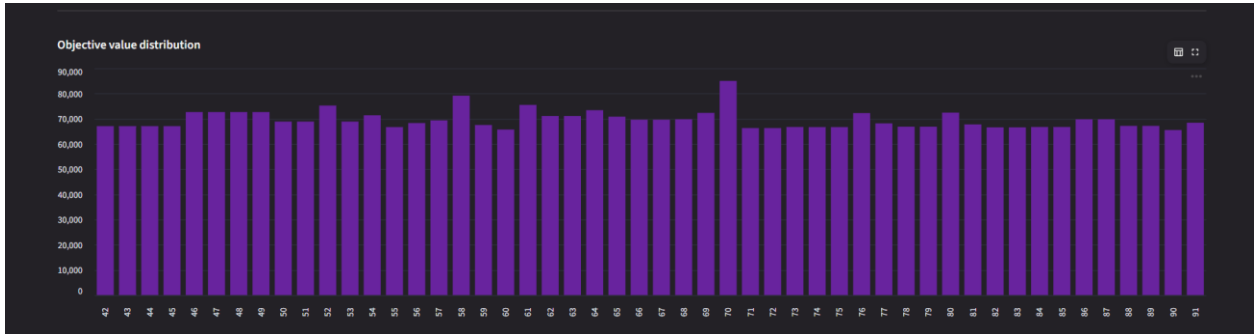


Figure 23: Monte-Carlo results

6.4 Limitations and Known Issues

6.4.1 Testing-Process Limitations

There is no automated unit-test suite (e.g. `pytest`) exercising individual functions in isolation, correctness relies `fixture_generator.py` and the end-to-end checks in `validation.py`

6.4.2 Application Limitations

- Solver feasibility is not guaranteed for arbitrary future calendars: if FIFA dates, CAF fixtures, and the venue/rest rules leave too little usable calendar, even the loosest domain-fallback policy and the final-round rescue model can fail.
- The Streamlit file cache is keyed on (path, modification time); on a filesystem with coarse mtime resolution, two writes to the same output file within the same time-resolution window are a theoretical (though not observed) source of stale reads.
- There is no authentication and no persistent audit trail of who ran which seed and when — acceptable for the project's single-user scope, but a limitation if the tool were ever exposed to multiple concurrent users.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7 Conclusion

7.1 Summary of Achievements

The project set out to replace manual, spreadsheet-driven fixture scheduling for the Egyptian Premier League with a constraint-based optimizer, and delivered a working end-to-end system rather than a proof-of-concept notebook. Concretely, the finished application:

- Generates a complete, structurally valid 306-match double round-robin before the calendar solver ever runs.
- Assigns every match a concrete date, kickoff slot, and venue with a single CP-SAT model that enforces rest days, weekly load caps, daily/slot capacity, round ordering, tier-1 derby slot placement, and the final-round single-slot publication rule as hard constraints, while minimizing a weighted objective covering travel distance, round-order drift, kickoff-time preference, and venue-relief costs.
- Enforces the CAF continental-competition rest-day rule by excluding conflicting slots from a match's domain.
- Recovers automatically from an infeasible strict solve through a three-tier domain-fallback retry and a dedicated final-round rescue model.
- Independently re-validates every hard rule after the solve and writes a human-readable pass/fail report.
- Ships as a complete interactive Streamlit dashboard ,so a non-technical user can run the optimizer, inspect the result, and download every artifact without touching code.
- Benchmarks its own output against six real historical Egyptian Premier League seasons and, separately, across many random seeds via a Monte Carlo batch mode.

These pieces demonstrate that a constraint-programming approach is practical for a league of this size on ordinary hardware.

7.2 Lessons Learned

7.2.1 The Final Round Is a Source of Difficulty

Forcing all nine Round-34 matches onto one shared date and kickoff slot is a small requirement to state but a large one to satisfy, because it couples matches that would otherwise be scheduled independently. Treating the whole season, including Round 34, as one uniform CP-SAT model meant that a single hard-to-place final-round match could make the entire 306-match solve infeasible. Splitting the problem — solve Rounds 1–33 first, then solve a small, dedicated model for Round 34 with the same rules downgraded to penalized soft violations — was a direct response to watching this happen during development, and it is a reusable pattern: when one small, tightly-coupled part of a large constraint model is the recurring source of infeasibility, it is cheaper to carve it out into its own rescue model than to keep loosening the shared model's domains for everyone.

7.2.2 Real Data Is Messier Than the Model Assumes

A Huge share of implementation effort went into `data_loader.py`'s sheet-name matching and column validation, and into the `NAME_MAP` dictionaries that reconcile inconsistent club-name spellings across historical season CSVs. None of this is visible in the mathematical model of Chapter 3B, but all of it was necessary before that model could run on real spreadsheets at all. We learned that the constraint model is often the easiest part to get right on paper, and the ingestion and normalization layer around it is where a project's real fragility happens.

7.3 Future Enhancements

7.3.1 Multi-objective Pareto optimization

The current solver uses a single weighted-sum objective. A Pareto front approach would let stakeholders visually trade off travel distance vs. tier-mismatch vs. rest-day quality instead of manually tuning the weights in `constants.py`.

7.3.2 Metaheuristic hybridization

Combine CP-SAT with a local search metaheuristic (e.g., Simulated Annealing, Tabu Search) to escape local optima and improve solution quality within the same time budget.

7.3.3 Weather and Ramadan awareness

Incorporate seasonal calendar events (e.g., Ramadan, extreme summer heat) as soft or hard constraints that affect kickoff time preferences or blackout periods.

7.3.4 Sensitivity analysis

Systematically measure how changes to individual constraint parameters (e.g., MIN_REST_DAYS_LOCAL, MAX_MATCHES_PER_DAY) impact solution quality and feasibility.

7.3.5 API layer

Expose the pipeline as a REST API so external systems (league management platforms, broadcasting schedulers) can trigger optimization runs and retrieve results programmatically.

7.3.6 League-agnostic configuration

Refactor the hardcoded 18-team, 34-round structure into a fully configurable input, making the tool usable for other leagues (e.g., Saudi Pro League, MLS, other African leagues).

7.3.7 Export to iCal/Google Calendar

Allow teams or fans to export their team's schedule as .ics calendar files.

8 References

- [1] R. V. Rasmussen and M. A. Trick, "Round robin scheduling – a survey," *European Journal of Operational Research*, 2008.
- [2] Ahram online, "EPL approve new format for Egyptian Premier League next season," 2024. [Online]. Available: <https://english.ahram.org.eg/NewsContent/6/51/532092/Sports/Egyptian-Football/EPL-approve-new-format-for-Egyptian-Premier-League.aspx>.
- [3] D. Van Bulck, D. Goossens and F. Spieksma, "Integer programming models for round robin tournaments," *European Journal of Operational Research*, 2023.
- [4] Ahram online, "Worn-out Egypt players in 'critical zone' due to congested schedules," 2022. [Online]. Available: <https://english.ahram.org.eg/News/467780.aspx>.
- [5] P. Staats, "Generic Scheduling of Sports Tournaments," Utrecht University, Utrecht, 2009.
- [6] D. R. Goossens and F. C. R. Spieksma, "Soccer schedules in Europe: an overview," *Journal of Scheduling*, vol. 15, no. 5, pp. 641-651, 2012.
- [7] K. Easton, G. Nemhauser and M. Trick, "The traveling tournament problem description and benchmarks," *Principles and Practice of Constraint Programming*, pp. 580-584, 2001.
- [8] G. Durán, M. Guajardo and J. Wolf-Yadlin, "Scheduling the Chilean soccer league by integer programming," *Interfaces*, vol. 37, no. 6, pp. 539-552, 2007.